

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ВВЕДЕНИЕ	2
1 ПОСТАНОВКА ЗАДАЧИ	3
2 ВЫПОЛНЕНИЕ ЗАДАНИЯ	4
2.1. Анализ предметной области	4
2.2. Проектирование БД Веб-сайта	5
2.3. Модуль авторизации	7
2.4. Модуль пользователя	9
2.5. Модуль лотов и ставок	10
2.6. Модуль администратора	11
2.7. Контейнеризация	12
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСТОЧНИКОВ	14
ПРИЛОЖЕНИЕ А. Диаграммы базы данных	15
ПРИЛОЖЕНИЕ Б. Исходный код приложения	17
ПРИЛОЖЕНИЕ В. Исходный код приложения	25

ВВЕДЕНИЕ

Актуальность темы обусловлена высоким спросом онлайн-аукционов как удобного способа купли-продажи товаров, обеспечивающих прозрачность торгов, надёжное хранение данных и гибкое управление. Современные веб-технологии позволяют создать платформу, доступную для широкой аудитории, которая будет открывать новые возможности для участников аукционов.

Целью работы является разработка онлайн-платформы аукциона, предоставляющей регистрацию и авторизацию пользователей, создание и управление лотами, участие в торгах с предложением цены, просмотр истории ставок и управление покупками. Система должна корректно обрабатывать следующие сценарии: запрет ставок на свой лот, отмену последней неперебитой ставки, выкуп или отказ при победе. Для администраторов предусматривается управление категориями. Реализация выполняется на платформе .NET, ASP.NET Core и Entity Framework Core с СУБД PostgreSQL, с развёртыванием в Docker.

Для реализации необходимо решить следующие задачи: провести анализ предметной области; спроектировать и нормализовать базу данных; разработать серверную логику (регистрация, управление лотами, ставки, завершение торгов, администрирование категорий); создать пользовательские интерфейсы; реализовать фильтрацию, поиск и сортировку; разработать личный кабинет; обеспечить ролевой доступ; выполнить тестирование и развернуть в Docker.

1 ПОСТАНОВКА ЗАДАЧИ

В рамках производственной практики требуется разработать онлайн-платформу для проведения аукционов. Система позволяет зарегистрированным пользователям создавать лоты с указанием начальной цены, описания, категории и срока действия, а также участвовать в торгах, предлагая свою цену. Победителем признаётся участник, предложивший наибольшую цену на момент закрытия лота, после чего он может выкупить лот или отказаться от покупки.

Функциональные требования включают регистрацию и аутентификацию с разграничением ролей (пользователь и администратор), просмотр каталога с фильтрацией по категориям, поиском, сортировкой и фильтрацией по времени окончания. На странице лота отображаются полная информация и история ставок, пользователь может предложить цену с проверками (выше текущей, не на свой лот, не дублировать). Предусмотрена отмена последней неперебитой ставки. После завершения торгов победитель выбирает выкуп или отказ. В личном кабинете доступны созданные лоты (просмотр, редактирование, удаление), история ставок и покупок. Администратор управляет категориями (создание, редактирование, удаление).

Технические требования: работа в современных браузерах, сервер на ASP.NET Core с Entity Framework Core, СУБД PostgreSQL, контейнеризация Docker, клиентская часть на HTML, CSS, JavaScript с HTMX.

2 ВЫПОЛНЕНИЕ ЗАДАНИЯ

2.1. Анализ предметной области

Аукцион представляет собой способ купли-продажи товаров или услуг, при котором покупатели соревнуются за право приобретения лота, предлагая последовательно повышающиеся цены. Продавец выставляет товар с начальной стоимостью, а участники делают ставки, увеличивая цену до тех пор, пока не будет определён победитель – тот, кто предложил максимальную сумму на момент закрытия торгов. Классическая модель аукциона предполагает открытые торги, где все участники видят текущие предложения и могут реагировать на них, что создаёт конкурентную среду и позволяет продавцу получить наиболее выгодную цену.

Наиболее распространённым является аукцион, в котором участники последовательно повышают цену, а лот достаётся тому, кто сделал последнюю и самую высокую ставку. Именно эта модель легла в основу разрабатываемой онлайн-платформы. Важной особенностью аукционов является временное ограничение – торги длятся в течение заданного периода, по истечении которого дальнейшие ставки не принимаются, и победитель определяется автоматически.

Система должна гарантировать достоверность информации о текущей цене и оставшемся времени, чтобы все участники находились в равных условиях. Необходимо соблюдать правила, исключающие возможность ставок на собственные лоты, а также предотвращать дублирование предложений от одного пользователя. После завершения торгов должна быть реализована чёткая процедура выкупа или отказа, чтобы продавец и покупатель могли завершить сделку. Для удобства пользователей требуется эффективная система поиска и фильтрации лотов по категориям, цене и времени окончания, а также личный кабинет, позволяющий отслеживать активность и историю участия.

Таким образом, разрабатываемая платформа должна реализовать все перечисленные функции в рамках веб-приложения, обеспечивая удобный интерфейс как для продавцов, так и для покупателей.

2.2. Проектирование БД Веб-сайта

В качестве системы управления базами данных выбрана PostgreSQL. Данное решение обусловлено рядом преимуществ: PostgreSQL является объектно-реляционной СУБД с открытым исходным кодом, поддерживает расширенные типы данных, обеспечивает высокую надёжность и соответствие стандартам SQL. Она хорошо интегрируется с платформой .NET и Entity Framework Core, предоставляя богатые возможности для миграций и работы с большими объёмами данных. Кроме того, PostgreSQL поддерживает сложные запросы, транзакционность и механизмы конкурентного доступа, что критически важно для корректной обработки ставок в условиях одновременных запросов от множества пользователей.

Модель данных включает следующие основные сущности. Пользователь (User) является центральной сущностью. Каждый пользователь может создавать множество лотов, делать множество ставок и совершать множество покупок. Лот (Lot) представляет собой выставляемый на торги товар, содержит информацию о названии, количестве, начальной и текущей цене, времени окончания торгов, описании, городе, способах оплаты и доставки, а также статус и признак закрытия. Каждый лот обязательно принадлежит одному пользователю-продавцу и относится к одной категории (Tag). С каждым лотом может быть связано несколько изображений (LotImage). Ставка (Bid) фиксирует предложение цены конкретным пользователем за конкретный лот. Покупка (Purchase) сохраняет информацию о завершённой сделке, когда победитель торгов выкупает лот по зафиксированной цене. Категория (Tag) служит для классификации лотов и имеет уникальное название.

Была построена ER-диаграмма «сущность-связь» в нотации П. Чена, которая позволяет представить базу данных на концептуальном уровне и определить атрибуты и логические связи сущностей. Диаграмма изображена рисунке А.1 в Приложении А.

Разработанная база данных состоит из шести таблиц, объединённых внешними ключами и обеспечивающих хранение всей необходимой информации для функционирования интернет-магазина. Ниже перечислены основные сущности и их назначение.

Таблица «users» содержит поля: login (строковый, первичный ключ), email (уникальный, допускает NULL), password_hash (обязательное, хранит хешированный пароль), is_admin (логический, по умолчанию false).

Таблица «tags» включает поля id (целочисленный, первичный ключ, автоинкремент) и name (обязательное, уникальное).

Таблица «lots» является наиболее объемной: id (первичный ключ), user_login (внешний ключ к users.login, продавец), tag_id (внешний ключ к tags.id), title, count (количество товара), initial_price и current_price (начальная и текущая цена, хранятся как decimal), end_time (время окончания торгов в формате DateTimeOffset), description, city, payment_method и delivery_payment (строковые, определяются перечислениями), leader_login (внешний ключ к users.login, ссылка на текущего лидера, может быть NULL), closed (логический, признак завершения торгов), status (строковое, отражает состояние лота). На таблицу lots также созданы индексы для ускорения запросов по внешним ключам и полям leader_login, tag_id, user_login.

Таблица «lot_images» служит для хранения изображений лотов и включает id (первичный ключ), lot_id (внешний ключ к lots.id с каскадным удалением) и image_path (путь к файлу изображения). Связь с лотом организована один-ко-многим, при этом допускается наличие нескольких изображений для одного лота, а первое изображение по порядку используется как миниатюра.

Таблица «bids» фиксирует ставки и содержит id (первичный ключ), user_login (внешний ключ к users.login), lot_id (внешний ключ к lots.id) и price (десятичное значение предложенной цены). Обе внешние связи настроены с каскадным удалением, что гарантирует удаление ставок при удалении пользователя или лота. Для ускорения выборок созданы индексы по полям lot_id и user_login.

Таблица «purchases» хранит информацию о покупках: id (первичный ключ), user_login (внешний ключ к users.login), lot_id (внешний ключ к lots.id), locked_price (цена, по которой была совершена покупка). Аналогично ставкам, связи настроены с каскадным удалением и индексируются.

На рисунке A.2 в Приложении A показана полная атрибутивная модель базы данных в нотации IDEF1X.

Все таблицы были составлены на языке C# и сгенерированы с помощью миграций и обновлений Entity Framework Core. С дополнительными атрибутами на полях и классах, уточняющие их назначение, позволили сгенерировать гибкую и удобную в использовании схему.

Все внешние связи между таблицами обеспечивают целостность данных на уровне СУБД, а использование индексов по внешним ключам и уникальных ограничений (на email пользователя и имя категории) гарантирует производительность и отсутствие дублирования критически важных значений. В результате была получена логически упорядоченная структура базы данных, которая соответствует требованиям сайта.

2.3. Модуль авторизации

Модуль авторизации обеспечивает защищённый доступ к приложению, реализуя регистрацию, вход и управление сессией. Аутентификация построена на основе JWT-токенов, которые генерируются после успешной проверки учётных данных и сохраняются в cookies клиента. Токен содержит информацию о пользователе (логин, роль) и подписывается секретным ключом, что гарантирует его целостность и невозможность подделки. Для повышения безопасности доступ к защищённым маршрутам требует наличия валидного токена, который проверяется на каждом запросе через middleware. При истечении срока действия основного токена система автоматически пытается обновить его с использованием refresh-токена, хранящегося в отдельной cookie и в базе данных. Фоновая служба периодически очищает просроченные refresh-токены, предотвращая их накопление. Обновление происходит незаметно для пользователя: клиент получает новую пару токенов при запросе к специальному эндпоинту.

Регистрация и вход реализованы через стандартные HTML-формы. При регистрации нового пользователя система принимает два параметра: адрес электронной почты и пароль. Перед сохранением в базу данных пароль подвергается криптографической обработке. На первом этапе генерируется уникальная криптографическая соль (salt). Эта соль комбинируется с введенным пользователем паролем, после чего полученная строка многократно пропускается через алгоритм хеширования PBKDF2 с использованием SHA256. Полученный хеш вместе с солью сохраняется в таблице Users вместо исходного пароля. Когда пользователь отправляет запрос на аутентификацию, система извлекает из базы данных соль, соответствующую указанному email. Используя эту соль и предоставленный пароль, сервер выполняет идентичную процедуру хеширования с теми же параметрами. Сравнение полученного хеша с хранимым в базе значением происходит через константную по времени функцию, что исключает возможность проведения timing-атак. Только при полном совпадении хешей доступ считается подтвержденным.

Валидация форм выполняется как на клиентской стороне, так и на серверной: проверяется уникальность логина и почты, совпадение пароля и его подтверждения, а также заполненность обязательных полей. Для улучшения пользовательского опыта реализована динамическая проверка существования логина или почты через HTMX-запросы, которые выводят сообщения об ошибках без перезагрузки страницы. При успешной аутентификации пользователь перенаправляется на ранее сохранённый URL или на главную страницу.

Если токен отсутствует или истёк, а пользователь не успел обновить его, система автоматически перенаправляет с защищённых маршрутов на страницу входа, сохраняя при этом адрес запрошенной страницы для последующего возврата. Выход из системы осуществляется путём удаления токенов из cookies и очистки refresh-токена в базе.

Скриншоты страниц представлены в Приложении Б.

Таким образом реализован функционал регистрации и авторизации пользователя.

2.4. Модуль пользователя

Модуль пользователя предоставляет интерфейс для просмотра и управления личной информацией, а также доступ к данным о созданных лотах, ставках и покупках. Публичная часть модуля доступна всем посетителям и показывает логин пользователя и список его лотов (без возможности редактирования). Для авторизованного пользователя открывается полный личный кабинет, где он может увидеть свои лоты с возможностью перехода к редактированию или удалению, а также отдельные вкладки со списком своих ставок и покупок. Каждая вкладка загружается динамически с помощью HTMX, что обеспечивает быстрый отклик без полной перезагрузки страницы.

Через этот модуль пользователь может управлять созданными лотами: переходить на страницу редактирования или удалять лот. Удаление сопровождается физическим удалением файлов изображений с диска и записей из базы. Для удобства в личном кабинете отображается миниатюра каждого лота, его название и текущая цена. В разделе ставок пользователь видит все лоты, на которые он делал ставки, с указанием своей ставки и статуса: если он является лидером, отображается соответствующая метка; если его перебили – показывается текущая цена и уведомление. В разделе покупок перечислены лоты, которые пользователь выкупил (статус Purchased), с возможностью перехода к их странице. Кроме того, модуль предоставляет возможность смены пароля: пользователь вводит старый и новый пароль, после чего выполняется проверка старого пароля, и при успехе хеш обновляется. Все операции, кроме просмотра публичного профиля, требуют авторизации и проверки, что текущий пользователь имеет доступ к запрашиваемым данным (например, при просмотре чужих ставок или покупок возвращается ошибка). Модуль тесно взаимодействует с моделью лотов (LotsModel) для получения списков лотов пользователя и с базой данных для выполнения запросов к таблицам ставок и покупок.

Скриншоты страниц представлены в Приложении Б.

Таким образом реализован функционал доступа к личному кабинету пользователя.

2.5. Модуль лотов и ставок

Модуль лотов и ставок является самым большим, так как представляет основную бизнес логику приложения. Модуль поделен на две части: публичную и защищенную. В публичную часть входит возможность просматривать лоты и всю информацию о них. В защищенной части производится все манипуляции с лотами и открывается возможность делать ставки.

Публичная часть модуля предоставляет всем посетителям возможность просматривать каталог лотов с фильтрацией по категориям, поиском по названию, городу или продавцу, а также сортировкой по цене, названию и времени окончания. Для удобства навигации реализована пагинация. При переходе на страницу конкретного лота отображается полная информация о товаре, включая фотографии, описание, текущую цену, лидера торгов, оставшееся время, способ оплаты и условия доставки. Здесь же выводится история всех сделанных ставок в хронологическом порядке.

Защищённая часть модуля доступна только авторизованным пользователям. Она включает создание новых лотов, редактирование и удаление собственных лотов. При создании лота необходимо указать название, город, начальную цену, количество, категорию, дату окончания торгов, описание, способ оплаты и условия доставки, а также загрузить минимум одно изображение. Все поля проходят валидацию на серверной стороне: проверяется положительность цены и количества, будущая дата окончания, существование категории, корректность выбранных способов оплаты, наличие изображений. При редактировании лота можно изменить любые параметры, кроме начальной цены, при этом старые изображения удаляются с диска, а новые сохраняются.

Участие в торгах реализовано через форму на странице лота. Пользователь вводит сумму, которая должна быть строго больше текущей цены, после чего система проверяет, не является ли участник продавцом, и не делал ли он уже ставку на этот лот. При успешной проверке создаётся запись о ставке, обновляется текущая цена и назначается новый лидер. Если пользователь передумал, он может отменить свою последнюю неперебитую ставку, и тогда

цена возвращается к предыдущему значению, а лидером становится предыдущий участник. Все изменения цен и списка ставок отображаются динамически благодаря использованию HTMX, который позволяет обновлять только нужные фрагменты страницы без полной перезагрузки.

После наступления времени окончания торгов лот автоматически переводится в закрытое состояние, и статус меняется на «ожидание подтверждения покупки». Победитель (лидер) видит на странице лота кнопку «Оставить заказ», которая позволяет зафиксировать покупку по текущей цене – при этом создаётся запись в таблице покупок, а статус лота становится «Выкуплен». Либо победитель может отказаться от покупки, тогда статус меняется на «отказ». В обоих случаях лот считается завершённым и больше не участвует в торгах.

Для повышения надёжности и предотвращения конкурентных гонок все операции со ставками и изменением данных лота выполняются в рамках одной транзакции, а при возникновении ошибок пользователю выводится понятное сообщение.

Дополнительно реализованы две фоновых службы: первая периодически проверяет истёкшие лоты и переводит их в закрытое состояние, чтобы продавец и покупатель могли завершить сделку, а вторая находит и удаляет с диска изображения-сироты, которые были прикреплены к удалённому лоту.

Скриншоты страниц представлены в Приложении Б.

2.6. Модуль администратора

Модуль администратора предназначен для управления категориями лотов и доступен только пользователям с соответствующим флагом `is_admin`. На главной странице административной панели отображается список всех категорий с возможностью поиска по названию. Администратор может создавать новые категории, указывая уникальное имя, при этом проверяется отсутствие дубликатов. Для существующих категорий доступны редактирование (изменение названия) и удаление. При удалении категории связанные с ней лоты не удаляются, однако связь с категорией теряется, что может потребовать ручного переназначения. Все операции с категориями выполняются с использованием

НТМХ для динамического обновления списка без перезагрузки страницы. Интерфейс администратора интуитивно понятен и позволяет быстро поддерживать актуальную структуру каталога, что упрощает навигацию для конечных пользователей.

Скриншоты страниц представлены в Приложении Б.

2.7. Контейнеризация

Для обеспечения воспроизводимости окружения и упрощения развертывания реализована контейнеризация приложения с использованием Docker. Оркестрация контейнеров организована через Docker Compose, который объединяет два сервиса: сервер и базу данных.

Сборка выполняется одной командой `docker compose up --build`, что позволяет развернуть полную инфраструктуру приложения на любой системе с установленным Docker.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработана онлайн-платформа аукциона, реализованная в виде веб-приложения на платформе .NET с использованием ASP.NET Core MVC и Entity Framework Core, взаимодействующего с реляционной базой данных PostgreSQL. Клиентская часть построена на HTML, CSS и JavaScript с применением библиотеки HTMX, что позволило обеспечить динамическое обновление контента без полной перезагрузки страниц, сохраняя преимущества серверной отрисовки. Приложение развёртывается в контейнерах Docker, что гарантирует воспроизводимость среды и упрощает установку.

Ключевая функциональность охватывает регистрацию и аутентификацию пользователей на основе JWT-токенов с автоматическим обновлением сессии, создание и управление лотами (просмотр, фильтрация, поиск, сортировка, редактирование, удаление), участие в торгах с валидацией ставок (проверка превышения текущей цены, запрет ставок на свой лот, предотвращение дублирования), отмену последней неперебитой ставки, а также завершение торгов с выбором выкупа или отказа. Для пользователей реализован личный кабинет с просмотром созданных лотов, истории ставок и покупок; администраторы могут управлять категориями. Архитектурно проект демонстрирует применение принципов MVC, безопасное хранение данных, строгую валидацию входящих данных и транзакционную обработку операций со ставками. Разработанное решение соответствует функциональным требованиям и готово к практическому использованию.

СПИСОК ИСТОЧНИКОВ

- 1) Иванова Г.С. Технология программирования: учебник для вузов. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2003. – 320 с.
- 2) Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. 4-е изд. – СПб.: Питер, 2013. – 896 с.
- 3) Албахари, Джозеф, Албахари, Бен. C# 9.0. Карманный справочник. : Пер. с англ. – СПб. : ООО “Диалектика”, 2021. – 256 с.
- 4) .NET 6 - ASP.NET core MVC - (Add database - SQLITE) [Электронный ресурс]. URL - <https://youtu.be/S7SdtcIr28s> (дата обращения: 09.07.2025)
- 5) Entity Properties [Электронный ресурс]. URL - <https://learn.microsoft.com/en-us/ef/core/modeling/entity-properties?tabs=data-annotations%2Cwith-nrt>
- 6) Secure Your .NET API in 15 Minutes: JWT Authentication Tutorial [Электронный ресурс]. URL - <https://youtu.be/6DWJIyipxzw> (дата обращения: 10.07.2025)
- 7) Best Practices for Secure Password Hashing in .NET (Stop Storing Passwords in Plain Text!) [Электронный ресурс]. URL - <https://www.youtube.com/watch?v=J4ix8Mhi3rs> (дата обращения: 10.07.2025)
- 8) Docker Compose with .NET 8, PostgreSQL, and Redis (step by step) [Электронный ресурс]. URL - <https://www.youtube.com/watch?v=WQFx2m5Ub9M&t=557s> (дата обращения: 11.07.2025)
- 9) Easy Email Verification in .NET: FluentEmail + Papercut [Электронный ресурс]. URL - <https://www.youtube.com/watch?v=KtCjH-1iCIk> (дата обращения: 11.07.2025)
- 10) Background tasks with hosted services in ASP.NET Core [Электронный ресурс]. URL - <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-9.0&tabs=visual-studio> (дата обращения: 12.07.2025)

ПРИЛОЖЕНИЕ А. Диаграммы базы данных

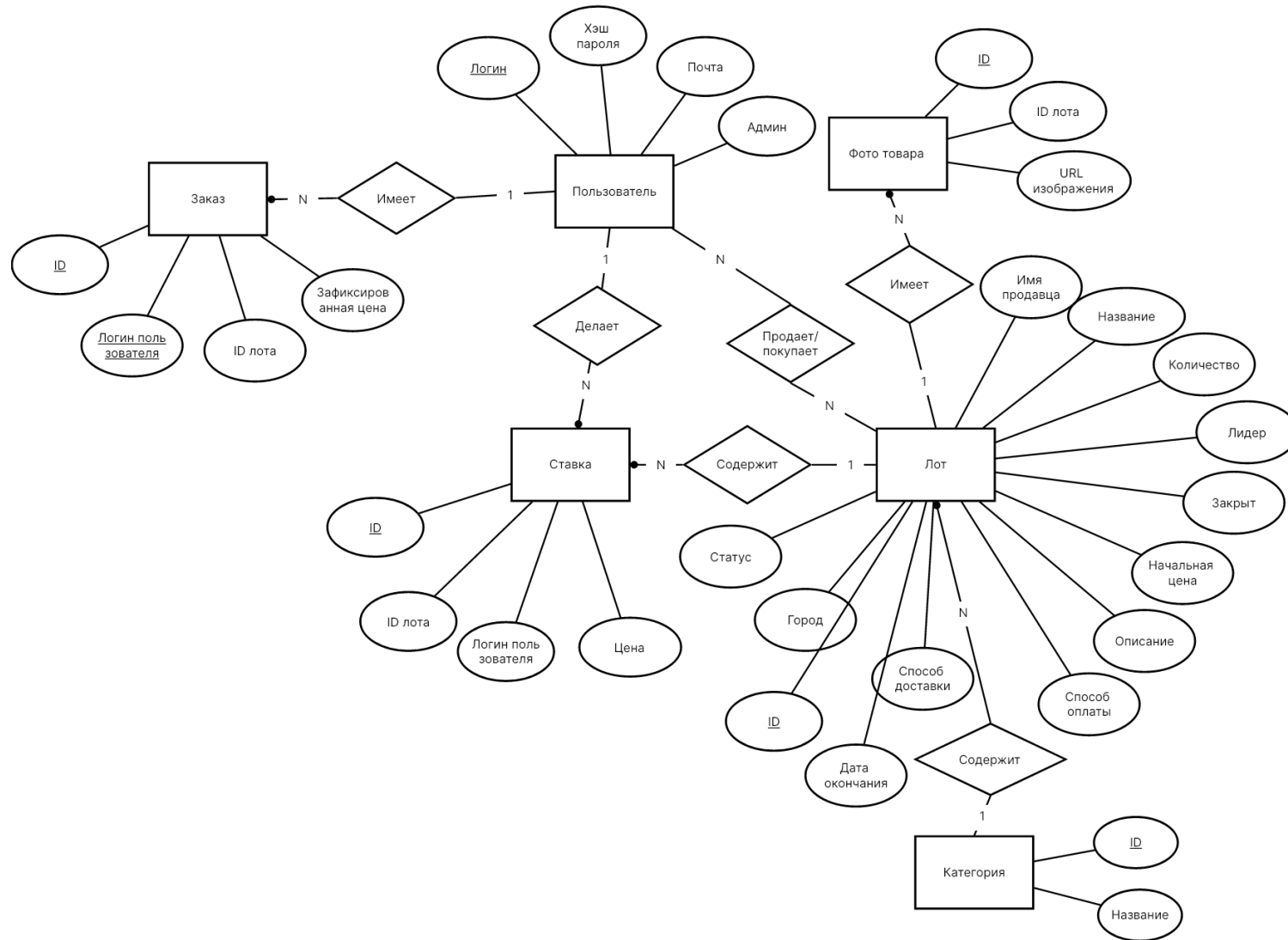


Рисунок А.1 – Диаграмма «сущность-связь»

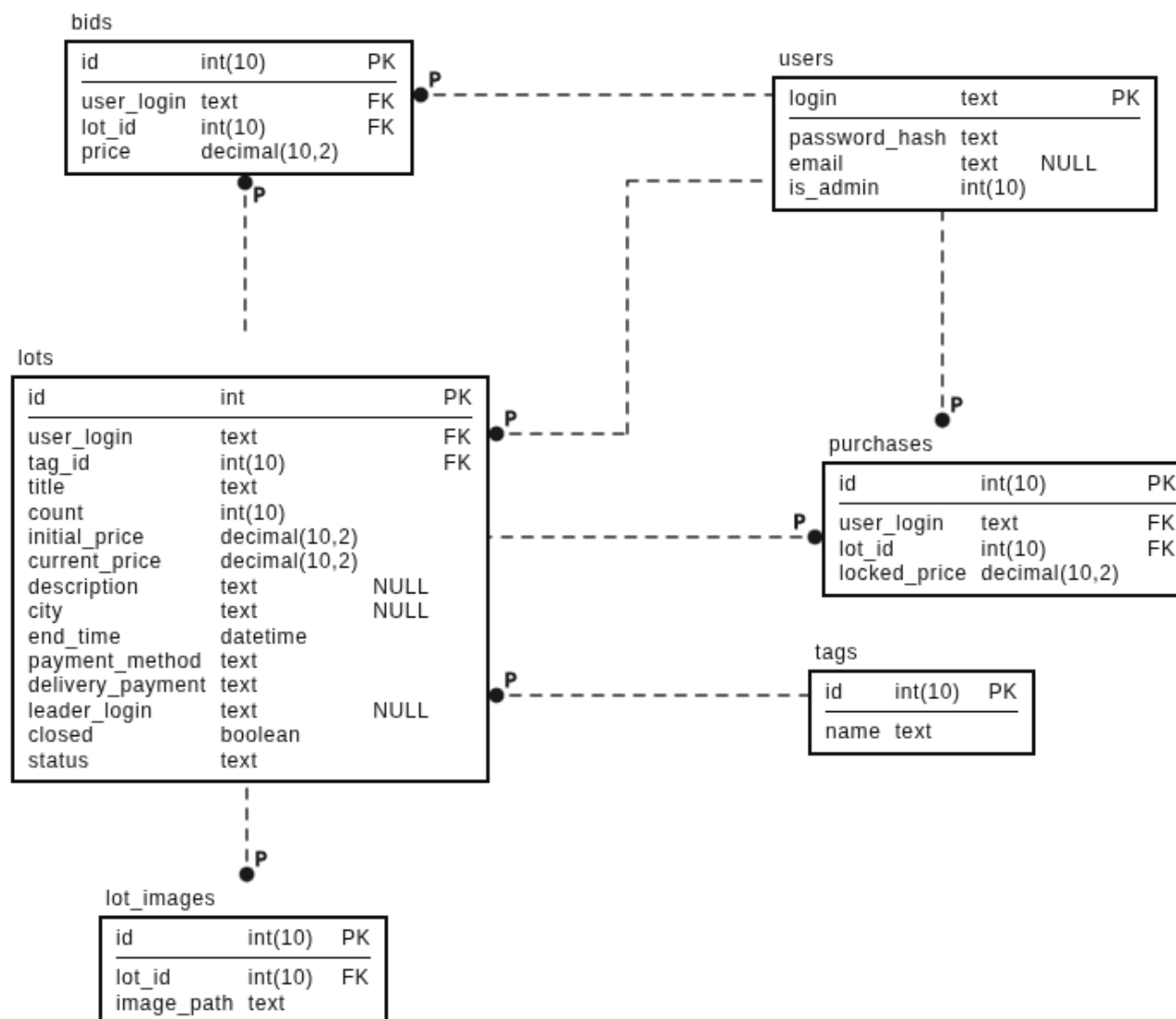
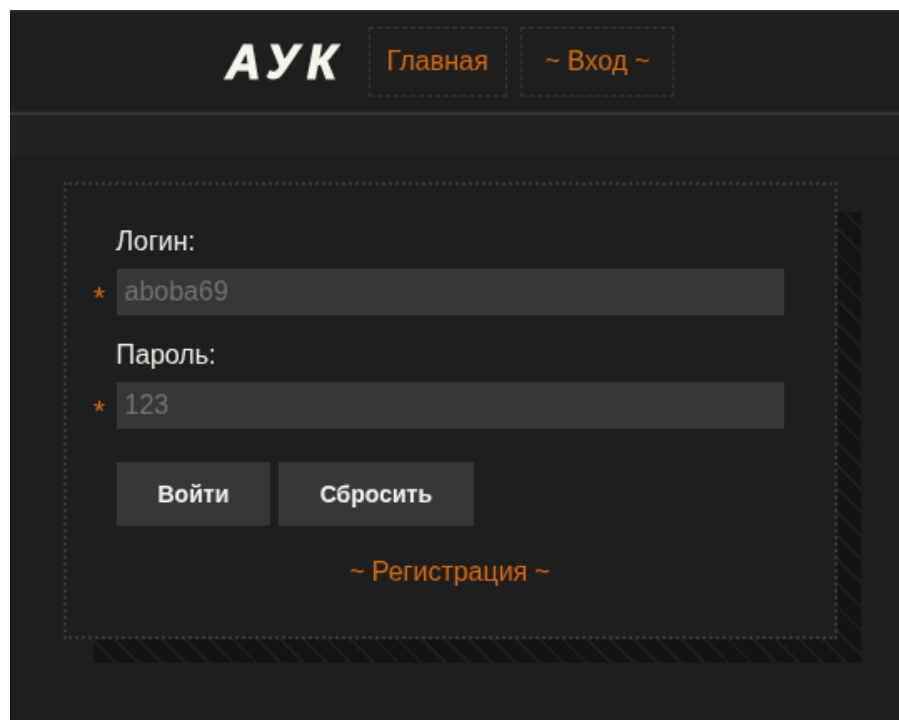


Рисунок А.2 – Полная атрибутивная модель

ПРИЛОЖЕНИЕ Б. Исходный код приложения



The image shows a login interface for an application named 'АУК'. At the top, there is a header bar with the 'АУК' logo on the left and two navigation links, 'Главная' and '~ Вход ~', on the right. Below the header is a central login form. The form contains two input fields: 'Логин:' with the value 'aboba69' and 'Пароль:' with the value '123'. Each input field has a small orange star icon to its left. Below the password field are two buttons: 'Войти' and 'Сбросить'. At the bottom of the form, there is a link '~ Регистрация ~'. The form is enclosed in a dashed border, and the background of the form area is a dark gray with a subtle diagonal line pattern.

АУК

Главная ~ Вход ~

Логин:

★ aboba69

Пароль:

★ 123

Войти Сбросить

~ Регистрация ~

Рисунок 3 – Форма входа

АУК

Главная

~ Вход ~

Логин:

★ aboba69

Почта:

urrtom@my.bed

Пароль:

★ 123

Подтвердите пароль:

★ 123

Регистрация

Сбросить

~ Вход ~

Рисунок 4 – Форма регистрации

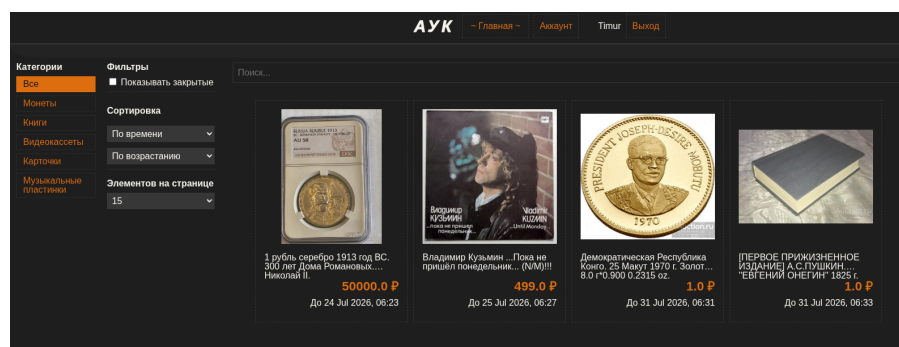


Рисунок 5 – Главная страница

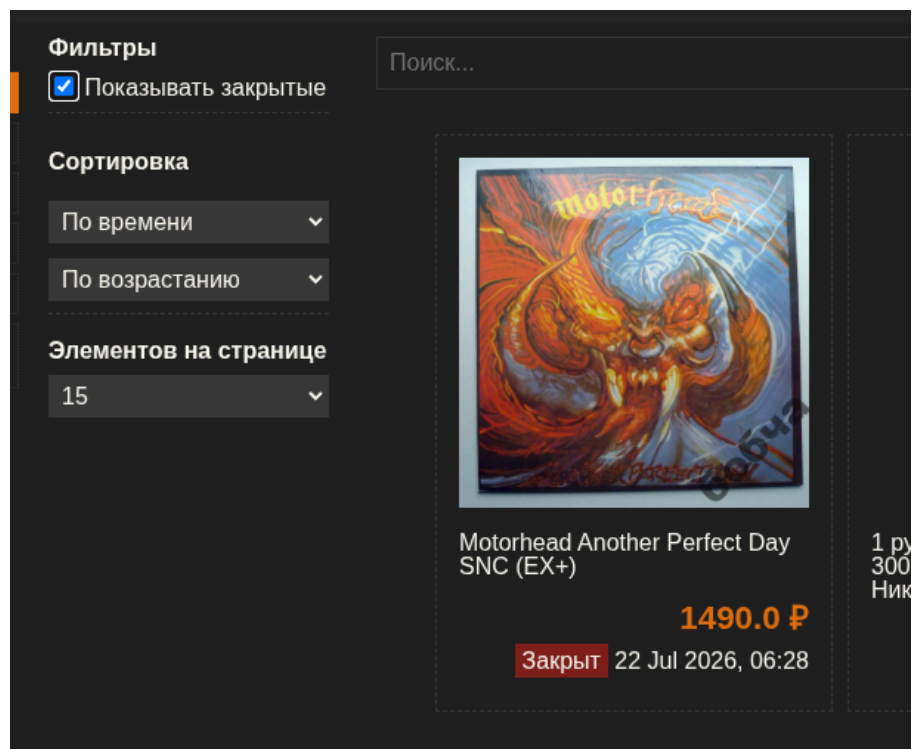


Рисунок 6 – Вывод закрыты лотов

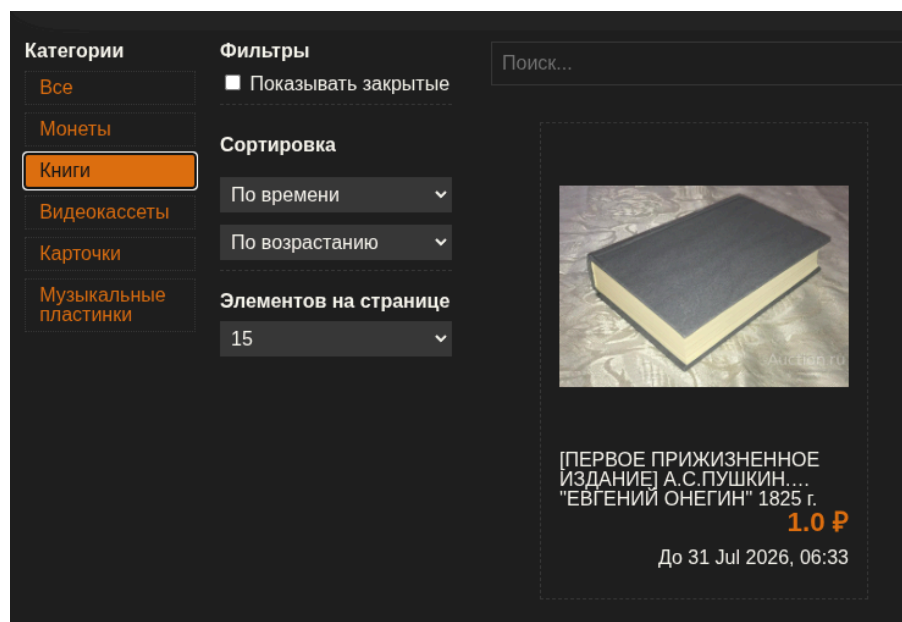


Рисунок 7 – Фильтр по категориям

Название
* 1 рубль серебро 1913 год ВС. 300 лет Дома Романовых. Николай II.

Город
* Москва

Цена
* 50000

Количество
* 1

Категория
* Монеты

Дата окончания
* 24.07.2026, 06:23

Описание
параметры:
Металл : серебро
Гарантия подлинности : Экспертное заключение, слаб
Номинал : Рубль

Рисунок 8 – Форма создания 1

Обложка

Выберите файл

1_rubl_serebro_1913_god_vs...00_proby_slab_cprc_au58.jpg

1_rubl_serebro_1913_god_vs_3...

62.2 КБ

Фото товара

Добавить файлы

Очистить файлы

1_rubl_serebro_1913_...

57.0 КБ

1_rubl_serebro_1913_...

61.3 КБ

Метод оплаты

★ При встрече

Оплата доставки

★ Покупатель

Создать

Очистить

Рисунок 9 – Форма создания 2

Логин: Timur

Почта: sdf@sdfmfn.sdf

Создать лот

Мои лоты

Мои ставки

Мои покупки

1 рубль серебро 1913 год ВС. 300 лет Дома Романовых. Николай II.

50000.0 Р

Удалить

Редактировать

Рисунок 10 – Созданные пользователем лоты

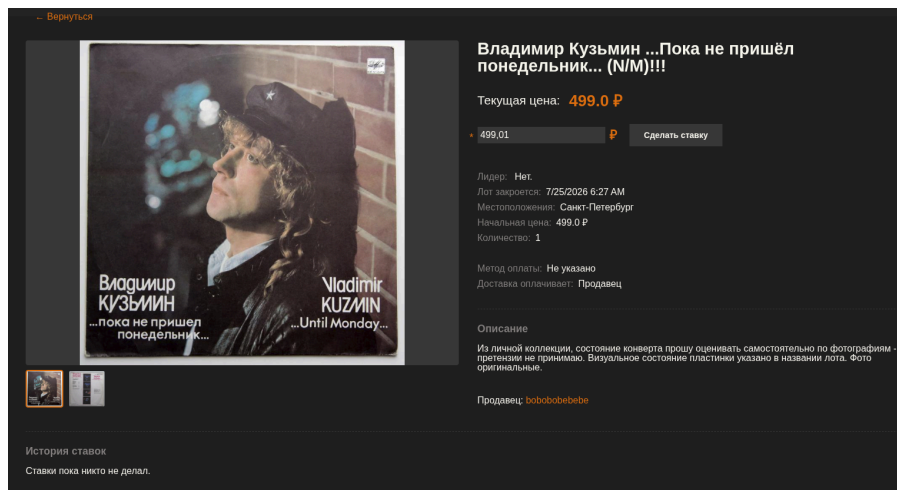


Рисунок 11 – Просмотр деталей лота

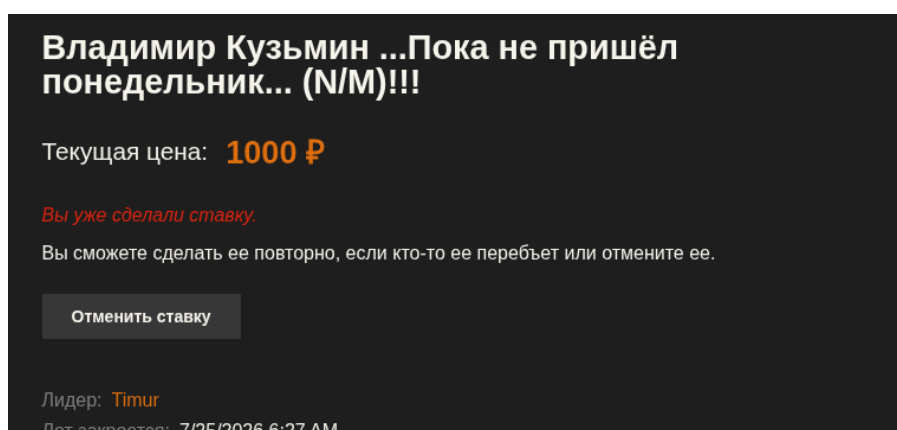


Рисунок 12 – Ставка сделана

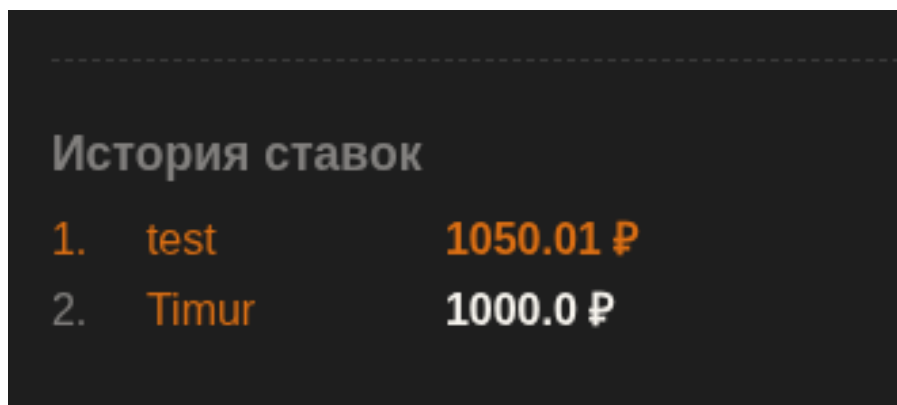


Рисунок 13 – История ставок

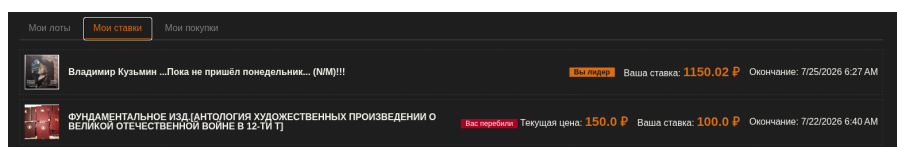


Рисунок 14 – Ставки пользователя

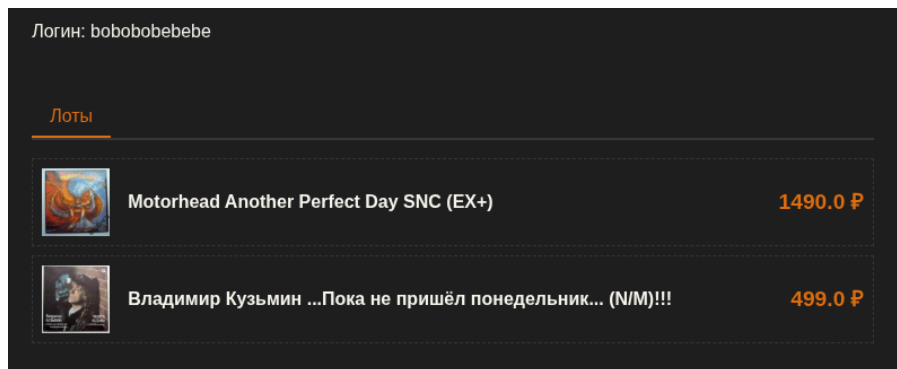


Рисунок 15 – Лоты другого пользователя

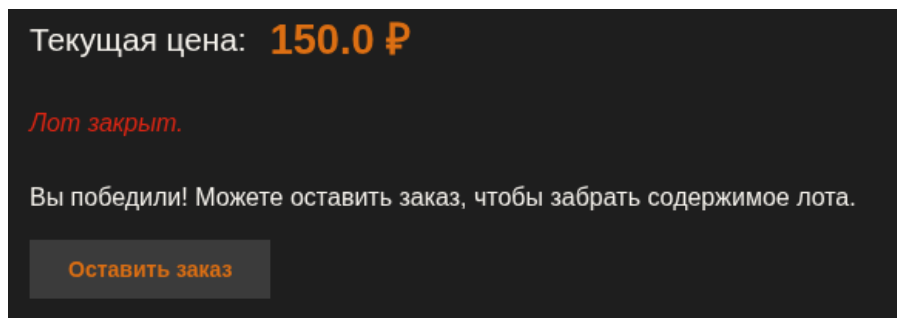


Рисунок 16 – Победа в торгах

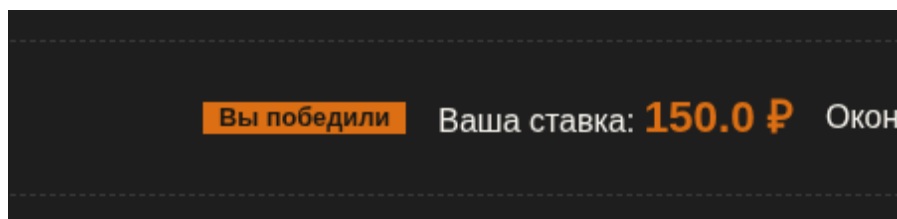


Рисунок 17 – Победа в торгах (в личном кабинете пользователя)

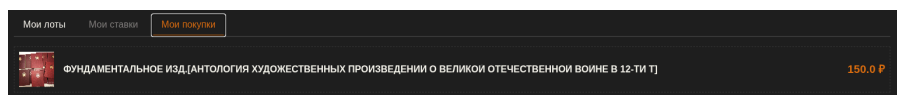


Рисунок 18 – Список покупок пользователя

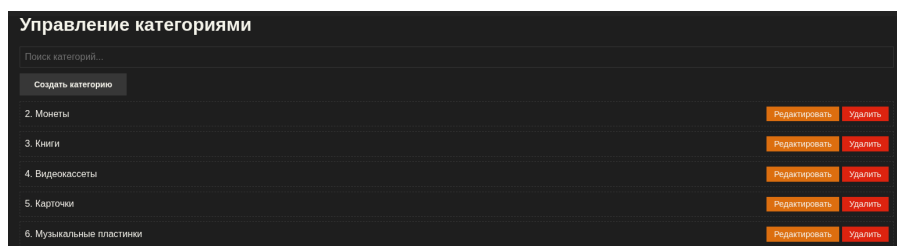


Рисунок 19 – Панель редактирования категорий

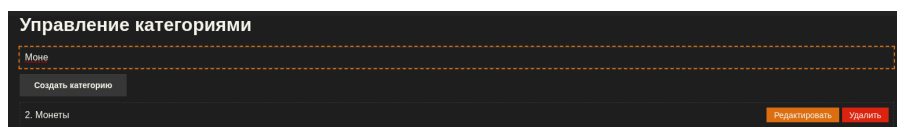
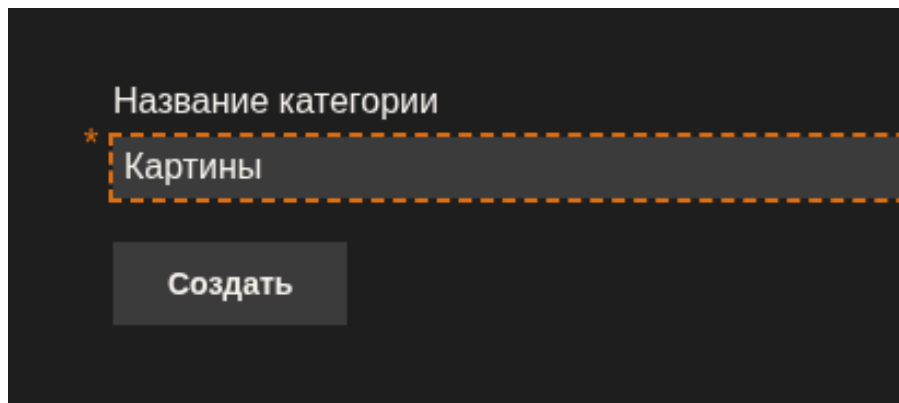


Рисунок 20 – Поиск категорий



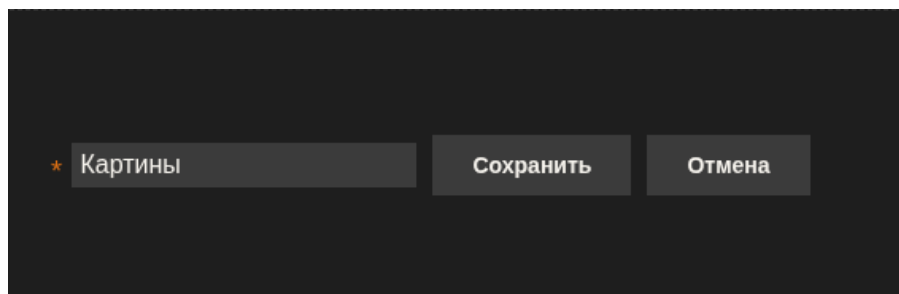
Название категории

* Картины

Создать

This screenshot shows a dark-themed user interface for creating a new category. At the top, the text 'Название категории' (Category Name) is displayed. Below it is a text input field containing the word 'Картины' (Paintings), preceded by an orange asterisk indicating a required field. The input field is outlined with a dashed orange border. Below the input field is a dark gray button with the text 'Создать' (Create) in white.

Рисунок 21 – Создание категории



* Картины

Сохранить

Отмена

This screenshot shows a dark-themed user interface for editing an existing category. It features a text input field containing the word 'Картины', preceded by an orange asterisk. To the right of the input field are two dark gray buttons: 'Сохранить' (Save) and 'Отмена' (Cancel), both with white text.

Рисунок 22 – Редактирование категории

ПРИЛОЖЕНИЕ В. Исходный код приложения

Листинг 1 – Файл Program.cs

```
using Microsoft.EntityFrameworkCore;
using App;
using App.Services;
using App.Models;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Authorization;
using System.Globalization;

CultureInfo.DefaultThreadCurrentCulture = new CultureInfo("en-US");

var builder = WebApplication.CreateBuilder(args);
var config = builder.Configuration;

builder.Services.AddControllers();
builder.Services.AddRazorPages();

builder.Services.AddDbContext<AuctionDbContext>(opt => {
    var conn_string = config.GetConnectionString("db_conn");
    if (builder.Environment.IsDevelopment()) {
        opt.UseSqlite(conn_string);
    } else {
        G.TODO("Add server db (like postgres or mysql) for production");
    }
});

builder.Services.AddScoped<JwtTokenService>();
builder.Services.AddHostedService<AuctionClosingService>();
builder.Services.AddHostedService<OrphanImagesKiller>();
builder.Services.AddScoped<PasswordHasher>();

builder.Services.AddAuthorizationBuilder()
    .SetDefaultPolicy(new AuthorizationPolicyBuilder()
        .AddAuthenticationSchemes(JwtBearerDefaults.AuthenticationScheme)
        .RequireAuthenticatedUser()
        .Build());

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => {
        o.RequireHttpsMetadata = false;
        o.TokenValidationParameters =
JwtTokenService.MakeTokenValidationParameters(config);
        o.Events = new JwtBearerEvents {
            OnMessageReceived = c => {
                var token_header =
c.Request.Headers["Authorization"].FirstOrDefault()?.Split(" ").Last();
                if (string.IsNullOrEmpty(token_header)) {
                    c.Token = c.Request.Cookies["jwt_token"];
                } else {
                    c.Token = token_header;
                }
            }
        }
    });
```

```

        return Task.CompletedTask;
    },
    OnChallenge = c => {
        var req_path = Uri.EscapeDataString(c.Request.Path.ToString());
        c.Response.Redirect($"/auth/login?saved_location={req_path}");
        c.HandleResponse();
        return Task.CompletedTask;
    }
};
});

var app = builder.Build();

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseStaticFiles();

app.MapControllers();

if (builder.Environment.IsDevelopment()) {
    using (var scope = app.Services.CreateScope()) {
        var db = scope.ServiceProvider.GetRequiredService<AuctionDbContext>();
        await db.Database.MigrateAsync();
    }
}

app.Run();

```

Листинг 2 – Файл register.cshtml

```

@using App.Controllers;
@using App.Extensions;
@{
    const string action = "/auth/register";
    const string login = "/auth/login";
    var last_saved_location = ViewData.GetSavedLocation();
}

<form id="register-form" class="form form-boxed shadow" method="post"
action="@action"
    hx-post="@action"
    hx-target="#error"
>
    <input name="last_saved_location" type="hidden"
value="@last_saved_location">

    <label for="login">Логин:</label>
    <div class="form-field">
        <input id="login" name="login" type="text" placeholder="aboba69"
required
            hx-post="/auth/register/user_exists"
            hx-trigger="input changed delay:200ms"
            hx-target="#error"
        />
    </div>

    <label for="email">Почта:</label>
    <div class="form-field">

```

```

        <input id="email" name="email" type="email" placeholder="urmom@my.bed"
            hx-post="/auth/register/user_exists"
            hx-trigger="input changed delay:200ms"
            hx-target="#error"
        />
    </div>

    <label for="password">Пароль:</label>
    <div class="form-field">
        <input id="password" name="password" type="password" placeholder="123"
required />
    </div>

    <label for="password_confirm">Подтвердите пароль:</label>
    <div class="form-field">
        <input id="password_confirm" name="password_confirm" type="password"
placeholder="123" required />
    </div>

    <div id="error">
        @await Html.PartialAsync("errors")
    </div>

    <button class="form-submit" type="submit">Регистрация</button>
    <button class="form-reset" type="reset">Сбросить</button>

    <p class="text-center"><a class="link-decor" href="@((login)?
saved_location=@(last_saved_location)"
        hx-get="@((login)?saved_location=@(last_saved_location)"
        hx-target="#register-form"
        hx-swap="outerHTML"
        hx-push-url="true"
    >Вход</a></p>
</form>

```

Листинг 3 – Файл Config.cs

```

namespace App;

public static class Config
{
    public const string APP_NAME = "Аук";
}

```

Листинг 4 – Файл auth.cs

```

using Microsoft.AspNetCore.Mvc;
using App.Attributes;
using App.Models;
using App.Services;
using App.Extentions;

namespace App.Controllers;

[ApiController]
[Route("auth")]
[Title("Авторизация")]
[HtmxServe, AddViewData]
[SaveLocation]

```

```

public class AuthController(
    PasswordHasher password_hasher,
    JwtTokenService token_service,
    AuctionDbContext db)
    : Controller
{
    AuthModel model = new AuthModel(db, password_hasher);

    [HttpGet("login")]
    public async Task<IActionResult> Login() {
        return View("login");
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromForm] User.LoginRequest login_req)
    {
        (User? user, AuthModel.Error err) = await
model.ValidateLoginForm(login_req);

        if (err != AuthModel.Error.None || user == null) {
            ViewData["errors"] = AuthModel.ErrorToString(err);
            if (Request.IsHtmx()) return View("errors");
            return View("login");
        }

        token_service.AppendTokenCookie(Response.Cookies, user);

        var url = "/";
        if (login_req.last_saved_location != null) {
            url = Uri.UnescapeDataString(login_req.last_saved_location);
        }
        return Redirect(url);
    }

    [HttpGet("register")]
    public async Task<IActionResult> Register() => View("register");

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromForm] User.RegisterRequest register_req)
    {
        (User? new_user, List<AuthModel.Error> errors) = await
model.RegisterNewUser(register_req);

        if (errors.Count > 0 || new_user == null) {
            ViewData["errors"] = errors.Select(AuthModel.ErrorToString);
            if (Request.IsHtmx()) return View("errors");
            return View("register");
        }

        token_service.AppendTokenCookie(Response.Cookies, new_user);

        var url = "/";
        if (register_req.last_saved_location != null) {
            url = Uri.UnescapeDataString(register_req.last_saved_location);
        }
        return Redirect(url);
    }
}

```

```

    }

    [Htmx]
    [HttpPost("login/user_exists")]
    public async Task<IActionResult> LoginCheckUserExists([FromForm] string?
login_or_email)
    {
        if (!await model.IsUserExists(login_or_email)) {
            ViewData["errors"] =
AuthModel.ErrorToString(AuthModel.Error.UserNotExists);
            return View("errors");
        }
        return Ok();
    }

    [Htmx]
    [HttpPost("register/user_exists")]
    public async Task<IActionResult> RegisterCheckUserExists([FromForm] string?
login, [FromForm] string? email)
    {
        var errors = await model.ValidateRegisterForm(new User.RegisterRequest(
            login ?? "",
            email
        ));

        if (errors.Count > 0) {
            ViewData["errors"] = errors.Select(AuthModel.ErrorToString);
            return View("errors");
        }

        return Ok();
    }

    [HttpPost("refresh")]
    public async Task<IActionResult> Refresh()
    {
        var token = Request.Cookies["jwt_token"];
        var refresh = Request.Cookies["jwt_refresh_token"];

        if (string.IsNullOrEmpty(token) || string.IsNullOrEmpty(refresh)) {
            return Unauthorized("No jwt_token or jwt_refresh_token");
        }

        var user_login = await token_service.ValidateRefreshToken(token,
refresh);
        if (user_login == null) {
            return Unauthorized("");
        }

        token_service.RemoveRefreshToken(refresh);

        var user = await db.GetUserByLogin(user_login);
        if (user == null) {
            return NotFound();
        }
        token_service.AppendTokenCookie(Response.Cookies, user);

        return Ok();
    }

```

```

    }

    [HttpPost("logout")]
    public async Task<IActionResult> Logout()
    {
        var token = Request.Cookies["jwt_token"];
        var refresh = Request.Cookies["jwt_refresh_token"];

        Response.Cookies.Delete("jwt_token");
        Response.Cookies.Delete("jwt_refresh_token");

        if (!string.IsNullOrEmpty(refresh)) {
            token_service.RemoveRefreshToken(refresh);
        }

        return Redirect(ViewData.GetSavedLocation());
    }
}

```

Листинг 5 – Файл user.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authorization;
using App.Models;
using App.Attributes;
using App.Extentions;
using App.Services;
namespace App.Controllers;

[ApiController]
[Route("user")]
[AddViewData, HtmxServe, GetUser]
public class PublicUserController(
    AuctionDbContext db,
    IWebHostEnvironment env
) : Controller
{
    private LotsModel lots_model = new(db, env);

    [HttpGet, Authorize, GetUserStrict]
    public IActionResult MyUserInfo()
    {
        var user = HttpContext.GetUser();
        return Redirect($"/user/{user.login}");
    }

    [HttpGet("{user_login}")]
    public async Task<IActionResult> UserInfo(string user_login)
    {
        var user = await db.GetUserByLogin(user_login);
        if (user == null) {
            return NotFound();
        }
        var lots = await lots_model.GetUserLots(user.login);
        var data = new UserInfoData {
            user_login = user.login,
            lots = lots
        }
    }
}

```

```

        };
        return View("User/user_info", data);
    }

    [HttpGet("{user_login}/lots-list")]
    public async Task<IActionResult> GetUserLotsList(string user_login)
    {
        var lots = await lots_model.GetUserLots(user_login);
        var data = new UserInfoData {
            user_login = user_login,
            lots = lots,
        };
        return View("User/user_lots", data);
    }
}

[ApiController]
[Route("user/{user_login}")]
[Authorize]
[GetUserStrict, AddViewData, HtmxServe]
public class UserController(
    AuctionDbContext db,
    PasswordHasher ph,
    IWebHostEnvironment env
) : Controller
{
    private LotsModel lots_model = new(db, env);

    [HttpGet("password_change")]
    public async Task<IActionResult> ChangePasswordForm(string user_login)
    {
        return View("change_password_form");
    }

    [HttpPatch("password_change")]
    public async Task<IActionResult> ChangePassword(string user_login,
    [FromForm] string old_password, [FromForm] string new_password)
    {
        var user = HttpContext.GetUser()!;
        if (user.login != user_login) {
            return Unauthorized();
        }

        if (ph.Verify(old_password, user.password_hash)) {
            return View("change_password_form", "Неправильный пароль.");
        }

        var new_hash = ph.Hash(new_password);
        user.password_hash = new_hash;
        db.Update(user);
        await db.SaveChangesAsync();

        return Redirect("/user");
    }

    [HttpGet("bets-list")]
    public async Task<IActionResult> GetUserBetsList(string user_login)
    {

```

```

        var user = HttpContext.GetUser()!;
        if (user.login != user_login) {
            return Unauthorized();
        }
        var bets = await db.bids
            .Where(b => b.user_login == user.login)
            .OrderByDescending(b => b.id)
            .Include(b => b.lot)
            .ThenInclude(l => l!.images)
            .ToListAsync();

        bets.Sort((b1, b2) => b2.lot!.closed.CompareTo(b1.lot!.closed));

        return View("user_bets", bets);
    }

    [HttpGet("purchases-list")]
    public async Task<IActionResult> GetUserPurchasesList(string user_login)
    {
        var user = HttpContext.GetUser()!;
        if (user.login != user_login) {
            return Unauthorized();
        }

        var purchases = await db.purchases
            .Where(p => p.user_login == user_login)
            .OrderByDescending(p => p.id)
            .Include(p => p.lot)
            .ThenInclude(l => l!.images)
            .ToListAsync();

        return View("user_purchases", purchases);
    }
}

```

Листинг 6 – Файл lots.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authorization;
using App.Models;
using App.Extentions;
using App.Attributes;
namespace App.Controllers;

[ApiController]
[Route("lots")]
[AddViewData, HtmxServe]
[GetUser]
public class PublicLotsController(AuctionDbContext db, IWebHostEnvironment env) : Controller
{
    public LotsModel model = new(db, env);

    [HttpGet("{id}")]
    public async Task<IActionResult> LotDetails([FromRoute] uint id)
    {
        var lot = await model.GetById(id);
    }
}

```



```

        if (lot == null) return NotFound();

        var images = lot.images.ToList();
        var seller = lot.user;
        var current_user = HttpContext.GetUser();
        var is_owner = current_user?.login == lot.user_login;

        var vm = new LotDetailsViewModel {
            lot = lot,
            images = images,
            seller = seller,
            is_owner = is_owner
        };
        return View("lot_details", vm);
    }

    [HttpPost("{id}/bid")]
    [Authorize, GetUserStrict]
    public async Task<IActionResult> MakeBid([FromRoute] uint id, [FromForm]
decimal new_price)
    {
        var user = HttpContext.GetUser()!;
        var (lot, error) = await model.MakeBid(id, new_price, user.login);

        if (lot == null) {
            return NotFound();
        }

        var data = new LotDetailsViewModel {
            lot = lot,
            seller = lot.user,
            images = lot.images.ToList(),
            is_owner = user?.login == lot.user_login,
        };

        if (error != null) {
            ViewData["errors"] = error;
            return View("bid_maker", data);
        }

        data.price_changed = true;

        return View("bid_maker", data);
    }

    [HttpDelete("{lot_id}/bid")]
    public async Task<IActionResult> CancelLastBid([FromRoute] uint lot_id)
    {
        var user = HttpContext.GetUser()!;
        var (lot, error) = await model.CancelLastBid(lot_id, user.login);

        if (lot == null) {
            return NotFound();
        }

        var data = new LotDetailsViewModel {
            lot = lot,
            seller = lot.user,

```

```

        images = lot.images.ToList(),
        is_owner = user?.login == lot.user_login,
    };

    if (error != null || lot == null) {
        ViewData["errors"] = error;
        return View("bid_maker", data);
    }

    data.price_changed = true;

    return View("bid_maker", data);
}
}

[ApiController]
[Route("lots/my")]
[AddViewData, HtmxServe]
[Authorize, GetUserStrict]
public class UserLotsController(AuctionDbContext db, IWebHostEnvironment env) :
Controller
{
    public LotsModel model = new(db, env);

    [HttpGet]
    public async Task<IActionResult> GetUserLots([FromQuery] int? page,
[FromQuery] int? page_size, [FromQuery] uint? tag_id)
    {
        var lots = model.GetPage(tag_id, page ?? 0, page_size ??
LotsModel.DEFAULT_PAGE_SIZE);
        return Ok(lots);
    }

    [HttpGet("{lot_id}")]
    public async Task<IActionResult> GetLot(uint lot_id)
    {
        var lot = await db.lots.FindAsync(lot_id);
        if (lot == null) {
            return NotFound();
        }
        return Ok(lot);
    }

    [HttpGet("create")]
    public async Task<IActionResult> CreateForm()
    {
        var tags = await db.tags.ToListAsync();
        return View("create_form", new CreateFormData{ tags = tags });
    }

    [HttpPost("create")]
    public async Task<IActionResult> Create([FromForm] Lot.CreateRequest req)
    {
        var user = HttpContext.GetUser();
        var (lot, errors) = await model.CreateLot(req, user.login);

        if (errors.Count > 0) {
            var tags = await db.tags.ToListAsync();

```

```

        var form_data = new CreateFormData { tags = tags };
        ViewData["errors"] = errors;
        return View("create_form", form_data);
    }

    return Redirect("/user");
}

[HttpDelete("{lot_id}")]
public async Task<IActionResult> DeleteById(uint lot_id)
{
    var (deleted_lot, err) = await model.DeleteById(lot_id);
    if (err != ModelError.None || deleted_lot == null) {
        Response.StatusCode = 400;
        return Content(err.GetMessage());
    }
    return Ok();
}

[HttpPatch("{lot_id}")]
public async Task<IActionResult> UpdateById([FromRoute] uint lot_id,
[FromForm] Lot.EditRequest req)
{
    var (updated_lot, errors) = await model.UpdateById(lot_id, req);
    if (errors.Count > 0 || updated_lot == null) {
        ViewData["errors"] = errors;
        return View("errors");
    }
    ViewData["result_messages"] = "Лот успешно обновлен!";
    return View("good_results");
}

[HttpGet("{lot_id}/purchase")]
public async Task<IActionResult> PurchaseForm([FromRoute] uint lot_id)
{
    var lot = await model.GetById(lot_id);
    if (lot == null) {
        return NotFound();
    }
    return View("purchase_form", lot);
}

[HttpPost("{lot_id}/purchase-submit")]
public async Task<IActionResult> PurchaseSubmit([FromRoute] uint lot_id)
{
    var user = HttpContext.GetUser();
    var lot = await model.GetById(lot_id);
    if (lot == null) {
        return NotFound();
    }

    var purchase = new Purchase {
        user_login = user.login,
        lot_id = lot_id,
        locked_price = lot.current_price,
    };
    db.purchases.Add(purchase);
}

```

```

        lot.status = LotStatus.Purchased.ToString();

        if (!await db.TrySaveChangesAsync()) {
            ViewData["errors"] = "Ошибка сохранения.";
            return View("purchase_form", lot);
        }
        return Redirect("/user");
    }

    [HttpPost("{lot_id}/purchase-deny")]
    public async Task<IActionResult> PurchaseDeny([FromRoute] uint lot_id)
    {
        var lot = await model.GetById(lot_id);
        if (lot == null) {
            return NotFound();
        }

        lot.status = LotStatus.Denied.ToString();

        if (!await db.TrySaveChangesAsync()) {
            ViewData["errors"] = "Ошибка сохранения.";
            return View("purchase_form", lot);
        }

        return Redirect("/user");
    }

    [HttpGet("{lot_id}/edit")]
    public async Task<IActionResult> EditForm([FromRoute] uint lot_id)
    {
        var lot = await model.GetById(lot_id);
        if (lot == null) {
            return NotFound();
        }
        var tags = await db.tags.ToListAsync();

        var data = new CreateFormData {
            current_lot = lot,
            tags = tags,
        };

        return View("edit_form", data);
    }
}

```

Листинг 7 – Файл admin.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using App.Models;
using App.Attributes;
using App.Extentions;
namespace App.Controllers;

[ApiController, Route("admin")]
[Authorize, GetUserAdmin]
[HttpxServe, AddViewData]
[Title("Администрирование")]

```

```

public class AdminController(AuctionDbContext db) : Controller
{
    private TagsModel tags_model = new(db);

    [HttpGet]
    public async Task<IActionResult> GetPage([FromQuery] string? search)
    {
        var tags = await tags_model.GetAll(search);
        var model = new AdminPageModel { Tags = tags, Search = search };
        if (Request.IsHtmx()) {
            return View("tag_list", model);
        }
        return View("index", model);
    }

    [HttpPost]
    public async Task<IActionResult> CreateTag([FromForm] Tag.CreateRequest req,
[FromQuery] string? search)
    {
        var (tag, errors) = await tags_model.CreateTag(req);

        var model = new AdminPageModel { Search = search };

        if (errors.Count > 0 || tag == null) {
            ViewData["errors"] = errors;
            return View("tag_form", model);
        }

        ViewData["result_messages"] = "Категория успешно создана!";
        var tags = await tags_model.GetAll(search);
        model.Tags = tags;
        model.Success = true;
        return View("tag_form", model);
    }

    [HttpDelete("{tag_id}")]
    public async Task<IActionResult> DeleteTag([FromRoute] uint tag_id)
    {
        var (tag, err) = await tags_model.DeleteById(tag_id);
        if (err != ModelError.None || tag == null) {
            Response.StatusCode = 400;
            G.Log(LogLevel.Error, err.GetMessage());
            return Content(err.GetMessage());
        }
        return Ok();
    }

    [HttpPatch("{tag_id}")]
    public async Task<IActionResult> UpdateTag([FromRoute] uint tag_id,
[FromForm] Tag.UpdateRequest req, [FromQuery] string? search)
    {
        var (updated_tag, errors) = await tags_model.UpdateById(tag_id, req);
        if (errors.Count > 0 || updated_tag == null) {
            ViewData["errors"] = errors;
            return View("tag_edit_form", new AdminPageModel { Success = false,
Search = search });
        }
    }
}

```

```

        return View("tag_item", updated_tag);
    }

    [HttpGet("{tag_id}/edit-form")]
    public async Task<IActionResult> EditForm([FromRoute] uint tag_id)
    {
        var tag = await db.tags.FindAsync(tag_id);
        if (tag == null) return NotFound();
        var form_data = new TagEditForm {
            Name = tag.name,
            TagId = tag.id,
        };
        return View("tag_edit_form", form_data);
    }
}

```

Листинг 8 – Файл home.cs

```

using Microsoft.AspNetCore.Mvc;
using App.Models;
using App.Attributes;
using App.Extentions;
namespace App.Controllers;

[ApiController]
[Route("/")]
[GetUser, HtmxServe, AddViewData]
[Title("Главная страница")]
public class HomeController(AuctionDbContext db, IWebHostEnvironment env) :
    Controller
{
    public LotsModel lots_model = new(db, env);

    [HttpGet("lots"), HttpGet]
    public async Task<IActionResult> LotCards([FromQuery] HomePageQuery query)
    {
        if (query.page_size == 0) query = query with { page_size =
        LotsModel.DEFAULT_PAGE_SIZE };
        var home_page = await lots_model.HomePage(query);
        if (Request.IsHtmx()) {
            ViewData["pagination"] = true;
            return View("lot_cards_grid", home_page);
        } else {
            return View("index", home_page);
        }
    }
}

```

Листинг 9 – Файл auth.cs

```

using App.Services;
namespace App.Models;

// TODO: Add checks for empty fields.
// The thing is that if client send an empty field which is not marked as
// nullable inside `RegisterRequest` or `LoginRequest` record, then the framework
// returns 415 automaticly. And this is a problem, because we can't show proper
// error message.
public class AuthModel(AuctionDbContext db, PasswordHasher password_hasher)

```

```

{
    public enum Error {
        None,
        UserNotExists,
        IncorrectPassword,
        LoginExists,
        EmailExists,
        PasswordConfirmNotMatch,
        DbError,
    }

    public static string ErrorToString(Error err)
    {
        return err switch {
            Error.None => "Нет ошибки.",
            Error.UserNotExists => "Пользователя не существует.",
            Error.IncorrectPassword => "Неправильный пароль.",
            Error.LoginExists => "Логин существует.",
            Error.EmailExists => "Почта существует.",
            Error.PasswordConfirmNotMatch => "Подтверждение пароля не
совпадает.",
            Error.DbError => "Пароли не совпадают.",
            _ => G.Unreachable<string>(nameof(ErrorToString)),
        };
    }

    public async Task<bool> IsUserExists(string? login_or_email)
    {
        if (login_or_email == null || login_or_email == "") return false;
        var user = await db.GetUserByLoginOrEmail(login_or_email);
        return user != null;
    }

    public async Task<(User? user, Error err)>
ValidateLoginForm(User.LoginRequest login_req)
    {
        var user = await db.GetUserByLoginOrEmail(login_req.login_or_email);
        if (user == null) {
            return (null, Error.UserNotExists);
        }

        if (!password_hasher.Verify(login_req.password, user.password_hash)) {
            return (null, Error.IncorrectPassword);
        }

        return (user, Error.None);
    }

    public async Task<List<Error>> ValidateRegisterForm(User.RegisterRequest
register_req)
    {
        var errors = new List<Error>();
        // if (register_req.login == null) {
        //     errors.Add("Логин не может быть пустым.");
        //     return errors;
        // }
        if (await db.GetUserByLogin(register_req.login) != null) {
            errors.Add(Error.LoginExists);
        }
    }
}

```

```

    }
    if (register_req.email != null && await
db.GetUserByEmail(register_req.email) != null) {
        errors.Add(Error.EmailExists);
    }
    if (register_req.password != register_req.password_confirm) {
        errors.Add(Error.PasswordConfirmNotMatch);
    }
    return errors;
}

public async Task<(User? new_user, List<Error> errors)>
RegisterNewUser(User.RegisterRequest register_req)
{
    var errors = await ValidateRegisterForm(register_req);

    if (errors.Count > 0) {
        return (null, errors);
    }

    var password_hash = password_hasher.Hash(register_req.password);
    var new_user = new User{
        login = register_req.login,
        email = register_req.email,
        password_hash = password_hash,
    };
    db.users.Add(new_user);

    if (!await db.TrySaveChangesAsync()) {
        errors.Add(Error.DbError);
        return (null, errors);
    }

    return (new_user, new());
}
}

```

Листинг 10 – Файл db_context.cs

```

using Microsoft.EntityFrameworkCore;
using System.Security.Claims;
namespace App.Models;

public enum ModelError {
    None,
    DbError,
    NotExist,
}

public static class DeleteErrorMethod {
    extension(ModelError err) {
        public string GetMessage()
        {
            return err switch {
                ModelError.None => "Нет ошибки.",
                ModelError.DbError => "Ошибка записи в базу данных.",
                ModelError.NotExist => "Записи не существует.",
                _ => G.Unreachable<string>(nameof(ModelError)),
            };
        }
    }
}

```



```

        };
    }
}

public class AuctionDbContext(DbContextOptions opt) : DbContext(opt)
{
    public DbSet<User>      users      {get; set;} = null!;
    public DbSet<Lot>       lots       {get; set;} = null!;
    public DbSet<LotImage> lot_images {get; set;} = null!;
    public DbSet<Tag>       tags       {get; set;} = null!;
    public DbSet<Bid>       bids       {get; set;} = null!;
    public DbSet<Purchase> purchases {get; set;} = null!;

    public async Task<User?> GetUserByClaims(ClaimsPrincipal user_claims)
    {
        var login_claim = user_claims.FindFirst("user_login");
        if (login_claim == null || string.IsNullOrEmpty(login_claim.Value))
        {
            return null;
        }
        var user_login = login_claim.Value;
        var user = await GetUserByLogin(user_login);
        return user;
    }

    public async Task<User?> GetUserByLogin(string login)
    {
        return await users.FirstOrDefaultAsync(u => u.login == login);
    }

    public async Task<User?> GetUserByEmail(string email)
    {
        return await users.FirstOrDefaultAsync(u => u.email == email);
    }

    public async Task<User?> GetUserByLoginOrEmail(string login_or_email)
    {
        return await users.FirstOrDefaultAsync(u => u.login == login_or_email ||
u.email == login_or_email);
    }

    public async Task<bool> TrySaveChangesAsync()
    {
        try {
            await SaveChangesAsync();
        } catch (Exception e) {
            G.Log(LogLevel.Error, e.ToString());
            return false;
        }
        return true;
    }
}

```

Листинг 11 – Файл models.cs

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

```

```

using Microsoft.EntityFrameworkCore;
namespace App.Models;

[Index(nameof(email), IsUnique = true)]
public class User
{
    [Key]          public required string login {get; set;}
    [EmailAddress] public required string? email {get; set;}
    [Required]     public required string password_hash {get; set;}

    public bool is_admin {get; set;} = false;

    [InverseProperty(nameof(Lot.user))]
    public ICollection<Lot> lots {get; set;} = new List<Lot>();
    [InverseProperty(nameof(Bid.user))]
    public ICollection<Bid> bets {get; set;} = new List<Bid>();
    [InverseProperty(nameof(Purchase.user))]
    public ICollection<Purchase> purchases {get; set;} = new List<Purchase>();

    public record LoginRequest(string login_or_email, string password, string?
last_saved_location = null);
    public record RegisterRequest(string login, string? email, string password =
"", string password_confirm = "", string? last_saved_location = null);

    public record Dto(string login, string? email);
    public Dto ToDto() => new Dto(login, email);
}

public enum PaymentMethod
{
    None,
    OnMeeting,
    ViaBank,
}

public enum DeliveryPayment
{
    PaidBySeller,
    PaidByCustomer,
}

public enum LotStatus
{
    Playing,
    WaitingPurchaseConfirm,
    Purchased,
    Denied,
}

public class Lot
{
    [Key] public uint id {get; set;}

    [Required] public string user_login {get; set;} = null!;
    [Required] public uint tag_id {get; set;}

    [Required] public string title          {get; set;} = null!;
    public int count                        {get; set;} = 1;
}

```

```

        [Required] public decimal initial_price {get; set;}
        public decimal current_price {get; set;}

        [Required] public DateTimeOffset end_time {get; set;}

        public string? description {get; set;}
        public string? city {get; set;}
        [Required] public string payment_method {get; set;} =
PaymentMethod.None.ToString();
        [Required] public string delivery_payment {get; set;} =
DeliveryPayment.PaidByCustomer.ToString();

        public string? leader_login {get; set;}
        public bool closed {get; set;} = false;

        public string status {get; set;} = LotStatus.Playing.ToString();

        public bool UpdateClosed()
        {
            var now = DateTime.UtcNow;
            if (end_time <= now) {
                closed = true;
                status = LotStatus.WaitingPurchaseConfirm.ToString();
            }
            return closed;
        }

        [ForeignKey(nameof(leader_login))]
        public User? leader {get; set;}
        [ForeignKey(nameof(user_login))]
        public User? user {get; set;}
        [ForeignKey(nameof(tag_id))]
        public Tag? tag {get; set;}

        [InverseProperty(nameof(LotImage.lot))]
        public ICollection<LotImage> images {get; set;} = new List<LotImage>();

        public LotImage? FirstImage()
        {
            return images.OrderBy(i => i.id).FirstOrDefault();
        }

        [InverseProperty(nameof(Bid.lot))]
        public ICollection<Bid> bids {get; set;} = new List<Bid>();

        public record CreateRequest(
            string title,
            string? description,
            string city,
            string payment_method,
            string delivery_payment,
            int count,
            decimal price,
            DateTime end_time,
            uint tag_id,
            IFormFile? thumbnail,
            IFormFileCollection? images
        );

```

```

public record EditRequest(
    string? title,
    string? description,
    string? city,
    string? payment_method,
    string? delivery_payment,
    int? count,
    DateTime? end_time,
    uint? tag_id,
    IFormFile? thumbnail,
    IFormFileCollection? images
);

public static Lot From(CreateRequest data, string user_login)
{
    var lot = new Lot {
        title = data.title,
        user_login = user_login,
        tag_id = data.tag_id,
        count = data.count,
        initial_price = data.price,
        current_price = data.price,
        end_time = data.end_time,
        description = data.description,
        city = data.city,
        payment_method = data.payment_method,
        delivery_payment = data.delivery_payment,
    };
    return lot;
}

public void Update(EditRequest data)
{
    title = data.title!;
    tag_id = (uint)data.tag_id!;
    count = (int)data.count!;
    end_time = (DateTimeOffset)data.end_time!;
    description = data.description;
    city = data.city;
    payment_method = data.payment_method!;
    delivery_payment = data.delivery_payment!;
}

public decimal RecalculatePrice()
{
    var last_bet = bids.Where(b => b.lot_id == id)
        .OrderByDescending(b => b.id)
        .FirstOrDefault();
    if (last_bet == null) {
        current_price = initial_price;
        return current_price;
    }
    current_price = last_bet.price;
    return current_price;
}
}

```

```

public class LotImage
{
    [Key] public uint id {get; set;}
    [ForeignKey(nameof(Lot))] public uint lot_id {get; set;}
    [Required] public string image_path {get; set;} = null!;

    [ForeignKey(nameof(lot_id))]
    public Lot? lot {get; set;}
}

[Index(nameof(name), IsUnique = true)]
public class Tag
{
    [Key] public uint id {get; set;}
    [Required] public string name {get; set;} = null!;

    public record CreateRequest(string name);
    public record UpdateRequest(string name);

    public static Tag From(CreateRequest req)
    {
        return new Tag {
            name = req.name,
        };
    }

    [InverseProperty(nameof(Lot.tag))]
    public ICollection<Lot> lots {get; set;} = new List<Lot>();
}

public class Bid
{
    [Key] public uint id {get; set;}
    [Required] public required string user_login {get; set;}
    [Required] public uint lot_id {get; set;}

    [Required] public decimal price {get; set;}

    [ForeignKey(nameof(user_login))]
    public User? user {get; set;}
    [ForeignKey(nameof(lot_id))]
    public Lot? lot {get; set;}
}

public class Purchase
{
    [Key] public uint id {get; set;}
    [Required] public required string user_login {get; set;}
    [Required] public uint lot_id {get; set;}

    [Required] public decimal locked_price {get; set;}

    [ForeignKey(nameof(user_login))]
    public User? user {get; set;}
    [ForeignKey(nameof(lot_id))]
    public Lot? lot {get; set;}
}

```

Листинг 12 – Файл tags.cs

```
using Microsoft.EntityFrameworkCore;
namespace App.Models;

public class TagsModel(AuctionDbContext db)
{
    public async Task<List<Tag>> GetAll(string? search)
    {
        var q = db.tags.AsQueryable();
        if (!string.IsNullOrEmpty(search)) {
            q = q.Where(t => t.name.Contains(search));
        }
        return await q.OrderBy(t => t.id).ToListAsync();
    }

    public async Task<(Tag? tag, List<string> errors)>
    CreateTag(Tag.CreateRequest req)
    {
        var errors = new List<string>();
        if (string.IsNullOrEmpty(req.name)) {
            errors.Add("Название категории не может быть пустым.");
        }

        var exists = await db.tags.AnyAsync(t => t.name == req.name);
        if (exists) {
            errors.Add("Категория с таким названием уже существует.");
        }

        if (errors.Count > 0) {
            return (null, errors);
        }

        var tag = Tag.From(req);
        db.tags.Add(tag);
        if (!await db.TrySaveChangesAsync()) {
            errors.Add("Ошибка сохранения категории.");
        }

        return (tag, errors);
    }

    public async Task<(Tag?, ModelError)> DeleteById(uint id)
    {
        var tag = await db.tags.FindAsync(id);
        if (tag == null) {
            return (null, ModelError.NotExist);
        }
        db.tags.Remove(tag);
        if (!await db.TrySaveChangesAsync()) {
            return (null, ModelError.DbError);
        }
        return (tag, ModelError.None);
    }

    public async Task<(Tag? updatedTag, List<string> errors)> UpdateById(uint
id, Tag.UpdateRequest req)
    {

```

```

        var errors = new List<string>();
        var tag = await db.tags.FindAsync(id);
        if (tag == null) {
            errors.Add("Категория не найдена."); return (null, errors);
        }

        if (string.IsNullOrEmpty(req.name)) {
            errors.Add("Название не может быть пустым.");
        } else {
            var exists = await db.tags.AnyAsync(t => t.name == req.name &&
t.id != id);
            if (exists) {
                errors.Add("Категория с таким названием уже существует.");
            }
        }

        if (errors.Count > 0) return (null, errors);

        tag.name = req.name;
        db.tags.Update(tag);
        if (!await db.TrySaveChangesAsync()) {
            errors.Add("Ошибка обновления.");
        }
        return (errors.Count == 0 ? tag : null, errors);
    }
}

```

Листинг 13 – Файл view_models.cs

```

namespace App.Models;

public record HomePageQuery(
    int page = 0,
    int page_size = LotsModel.DEFAULT_PAGE_SIZE,
    uint? tag_id = null,
    string? search = null,
    string sort_by = "end_time",
    string sort_dir = "asc",
    bool show_closed = false
);

public class HomePageModel
{
    public struct LotCard
    {
        public uint id;
        public string? image_path;
        public string title;
        public decimal price;
        public DateTimeOffset end_time;
        public bool closed;
    }

    public List<LotCard> cards = null!;
    public List<Tag> tags = null!;
    public HomePageQuery query = null!;
    public int total_count;
    public int pages_count;
}

```

```

}

public class CreateFormData
{
    public List<Tag> tags = new();
    public Lot? current_lot = null;
}

public class TagNavigation
{
    public uint id;
    public required string name;
}

public class LotDetailsViewModel
{
    public Lot lot = new();
    public List<LotImage> images = new();
    public User? seller;
    public bool is_owner;
    public bool price_changed = false;
}

public class UserInfoData
{
    public string user_login = null!;
    public List<UserLotCard> lots = new();
}

public class UserLotCard
{
    public uint id;
    public string title = null!;
    public decimal current_price;
    public string? thumbnail_path;
}

public class AdminPageModel
{
    public List<Tag> Tags { get; set; } = new();
    public string? Search { get; set; }
    public bool Success { get; set; }
}

public class TagEditForm
{
    public uint TagId { get; set; }
    public string Name {get; set;} = null!;
    public bool Success {get; set;}
}

```

Листинг 14 – Файл lots.cs

```

using Microsoft.EntityFrameworkCore;
namespace App.Models;

public class Aboba : ICloneable
{

```



```

    public Lot lot = null!;

    public object Clone()
    {
        return this.MemberwiseClone();
    }
}

public class LotsModel(AuctionDbContext db, IWebHostEnvironment env)
{
    public const int DEFAULT_PAGE_SIZE = 15;

    public async Task<Lot?> GetById(uint id)
    {
        // TODO: maybe separate into different methods or query view parameters
        var lot = await db.lots
            .Include(l => l.images)
            .Include(l => l.user)
            .Include(l => l.leader)
            .Include(l => l.bids.OrderByDescending(b => b.id))
            .FirstOrDefaultAsync(l => l.id == id);
        return lot;
    }

    public IQueryable<Lot> GetFilteredQuery(HomePageQuery query)
    {
        var q = db.lots.AsQueryable();

        if (query.tag_id.HasValue) {
            q = q.Where(l => l.tag_id == query.tag_id.Value);
        }

        if (!string.IsNullOrEmpty(query.search)) {
            var searchLower = query.search.ToLower();
            q = q.Where(l =>
                l.title.ToLower().Contains(searchLower) ||
                (l.city != null && l.city.ToLower().Contains(searchLower)) ||
                l.user_login.ToLower().Contains(searchLower)
            );
        }

        if (!query.show_closed) {
            q = q.Where(l => !l.closed);
        }

        q = query.sort_by switch {
            "price" => query.sort_dir == "asc" ? q.OrderBy(l =>
                l.current_price) : q.OrderByDescending(l => l.current_price),
            "title" => query.sort_dir == "asc" ? q.OrderBy(l => l.title) :
                q.OrderByDescending(l => l.title),
            _ => query.sort_dir == "asc" ? q.OrderBy(l =>
                l.end_time.ToString()) : q.OrderByDescending(l => l.end_time.ToString()),
        };

        return q;
    }

    public async Task<List<HomePageModel.LotCard>> GetLotCardsPage(HomePageQuery

```

```

query)
{
    var lots = await GetFilteredQuery(query)
        .Skip(query.page * query.page_size)
        .Take(query.page_size)
        .Include(l => l.images)
        .ToListAsync();

    var cards = new List<HomePageModel.LotCard>();
    foreach (var lot in lots) {
        var image = lot.images.FirstOrDefault();
        cards.Add(new HomePageModel.LotCard {
            id = lot.id,
            image_path = image?.image_path,
            title = lot.title,
            price = lot.current_price,
            end_time = lot.end_time,
            closed = lot.closed,
        });
    }

    return cards;
}

public async Task<HomePageModel> HomePage(HomePageQuery query) {
    var total = await GetFilteredQuery(query).CountAsync();
    int pages = total == 0 ? 1 : (int)Math.Ceiling((double)total /
query.page_size);

    if (query.page >= pages) query = query with { page = pages - 1 };
    if (query.page < 0) query = query with { page = 0 };

    var tags = await db.tags.ToListAsync();
    var cards = await GetLotCardsPage(query);

    return new HomePageModel {
        cards = cards,
        tags = tags,
        query = query,
        total_count = total,
        pages_count = pages
    };
}

public IQueryable<Lot> GetPage(uint? tag_id, int page, int page_size =
DEFAULT_PAGE_SIZE) {
    var lots_query = tag_id switch {
        null => db.lots,
        _ => db.lots.Where(l => l.tag_id == tag_id)
    };
    return lots_query
        .Skip(page*page_size)
        .Take(page_size)
        .Include(l => l.images);
}

public async Task<(Lot? deleted_lot, ModelError err)> DeleteById(uint id)
{

```

```

        var lot = await GetById(id);
        if (lot == null) {
            return (null, ModelError.NotExist);
        }

        var images = lot.images;
        var uploads_path = Path.Combine(env.WebRootPath, "uploads",
"lot_images");
        await DeleteLotImagesFiles(images, uploads_path);

        db.lots.Remove(lot);
        if (!await db.TrySaveChangesAsync()) {
            return (null, ModelError.DbError);
        }

        return (lot, ModelError.None);
    }

    public async Task<(Lot? updated_lot, List<string> errors)> UpdateById(uint
id, Lot.EditRequest req)
    {
        var errors = Validate.AllEdit(req, db);
        if (errors.Count > 0) return (null, errors);

        var lot = await db.lots.Include(l => l.images).FirstOrDefaultAsync(l =>
l.id == id);
        if (lot == null) {
            errors.Add("Лота не существует.");
            return (null, errors);
        }

        lot.Update(req);

        var uploads_path = Path.Combine(env.WebRootPath, "uploads",
"lot_images");
        Directory.CreateDirectory(uploads_path);

        var images = new List<LotImage>(lot.images);

        db.lot_images.RemoveRange(lot.images);
        if (!await db.TrySaveChangesAsync()) {
            errors.Add("Ошибка сохранения в базе данных.");
            return (null, errors);
        }

        await DeleteLotImagesFiles(images, uploads_path);
        await SaveLotImagesFiles(lot, req.thumbnail, req.images, uploads_path);

        if (!await db.TrySaveChangesAsync()) {
            errors.Add("Ошибка сохранения в базе данных.");
            return (null, errors);
        }

        return (lot, errors);
    }

    public static async Task SaveLotImagesFiles(Lot lot, IFormFile? thumbnail,
IFormFileCollection? images, string uploads_path)

```

```

{
    var images_to_save = new List<IFormFile>();
    if (thumbnail != null) images_to_save.Add(thumbnail);
    if (images != null) images_to_save.AddRange(images);

    foreach (var file in images_to_save) {
        if (file.Length == 0) continue;
        var ext = Path.GetExtension(file.FileName);
        var file_name = $"{Guid.NewGuid()} {ext}";
        var file_path = Path.Combine(uploads_path, file_name);

        using (var stream = new FileStream(file_path, FileMode.Create)) {
            await file.CopyToAsync(stream);
        }

        var lot_image = new LotImage {
            image_path = $"/uploads/lot_images/{file_name}"
        };

        lot.images.Add(lot_image);
    }
}

public static async Task DeleteLotImagesFiles(ICollection<LotImage> images,
string uploads_path)
{
    foreach (var img in images) {
        var file_name = Path.GetFileName(img.image_path);
        var file_path = Path.Combine(uploads_path, file_name);
        if (System.IO.File.Exists(file_path)) {
            System.IO.File.Delete(file_path);
        }
    }
}

public async Task<(Lot? lot, List<string> errors)>
CreateLot(Lot.CreateRequest req, string user_login)
{
    var errors = Validate.AllCreate(req, db);
    if (errors.Count > 0) return (null, errors);

    var lot = Lot.From(req, user_login);

    var uploads_path = Path.Combine(env.WebRootPath, "uploads",
"lot_images");
    Directory.CreateDirectory(uploads_path);

    await SaveLotImagesFiles(lot, req.thumbnail, req.images, uploads_path);

    for (int i = 0; i < 30; i++) {
        db.lots.Add(lot);
    }
    if (!await db.TrySaveChangesAsync()) {
        errors.Add("Ошибка сохранения в базу данных.");
        foreach (var img in lot.images) {
            var file_name = Path.GetFileName(img.image_path);
            var file_path = Path.Combine(uploads_path, file_name);
            if (System.IO.File.Exists(file_path)) {

```

```

        System.IO.File.Delete(file_path);
    }
}
return (null, errors);
}

return (lot, errors);
}

public static class Validate
{
    public static List<string> AllCreate(Lot.CreateRequest req,
AuctionDbContext db)
    {
        var errors = new List<string>();
        Title(req.title, ref errors);
        Count(req.count, ref errors);
        Price(req.price, ref errors);
        EndTime(req.end_time, ref errors);
        TagExists(req.tag_id, db, ref errors);
        Images(req.thumbnail, req.images, ref errors);
        City(req.city, ref errors);
        PaymentMethod(req.payment_method, ref errors);
        DeliveryPayment(req.delivery_payment, ref errors);
        return errors;
    }

    public static List<string> AllEdit(Lot.EditRequest req, AuctionDbContext
db)
    {
        var errors = new List<string>();
        Title(req.title, ref errors);
        Count(req.count, ref errors);
        EndTime(req.end_time, ref errors);
        TagExists(req.tag_id, db, ref errors);
        Images(req.thumbnail, req.images, ref errors);
        City(req.city, ref errors);
        PaymentMethod(req.payment_method, ref errors);
        DeliveryPayment(req.delivery_payment, ref errors);
        return errors;
    }

    public static void Title(string? title, ref List<string> errors)
    {
        if (string.IsNullOrEmpty(title))
            errors.Add("Название не может быть пустым.");
    }

    public static void Count(int? count, ref List<string> errors)
    {
        if (count == null || count < 1)
            errors.Add("Количество должно быть не меньше 1.");
    }

    public static void Price(decimal? price, ref List<string> errors)
    {
        if (price == null || price <= 0)
            errors.Add("Цена должна быть положительной.");
    }
}

```

```

    }

    public static void EndTime(DateTimeOffset? end_time, ref List<string>
errors)
    {
        if (end_time == null || end_time <= DateTimeOffset.Now)
            errors.Add("Дата окончания должна быть в будущем.");
    }

    public static void TagExists(uint? tag_id, AuctionDbContext db, ref
List<string> errors)
    {
        const string msg = "Выбранная категория не существует.";

        if (tag_id == null) {
            errors.Add(msg);
            return;
        }

        var tag_exists = db.tags.Any(t => t.id == tag_id);
        if (!tag_exists) {
            errors.Add(msg);
            return;
        }
    }

    public static void Images(IFormFile? thumbnail, IFormFileCollection?
images, ref List<string> errors)
    {
        if (thumbnail == null && (images == null || images.Count == 0))
            errors.Add("Необходимо добавить хотя бы одно изображение.");
    }

    public static void City(string? city, ref List<string> errors)
    {
        if (string.IsNullOrEmpty(city))
            errors.Add("Необходимо указать город.");
    }

    public static void PaymentMethod(string? payment_method, ref
List<string> errors)
    {
        if (string.IsNullOrEmpty(payment_method)) {
            errors.Add("Необходимо указать способ оплаты.");
            return;
        }
        bool ok = false;
        foreach (var p in G.EnumIterate<PaymentMethod>()) {
            if (p.ToString() == payment_method) {
                ok = true;
                break;
            }
        }
        if (!ok) {
            errors.Add($"Способ оплаты не поддерживается.");
        }
    }
}

```

```

        public static void DeliveryPayment(string? delivery_payment, ref
List<string> errors)
        {
            if (string.IsNullOrEmpty(delivery_payment)) {
                errors.Add("Необходимо указать, кто оплачивает доставку.");
                return;
            }
            bool ok = false;
            foreach (var p in G.EnumIterate<DeliveryPayment>()) {
                if (p.ToString() == delivery_payment) {
                    ok = true;
                    break;
                }
            }
            if (!ok) {
                errors.Add($"Вариант оплаты {delivery_payment} не
поддерживается.");
            }
        }
    }

    public async Task<List<UserLotCard>> GetUserLots(string user_login)
    {
        var user = await db.users
            .Where(u => u.login == user_login)
            .Include(u => u.lots)
            .FirstOrDefaultAsync();

        if (user == null) {
            return new();
        }

        var cards = new List<UserLotCard>();
        foreach (var lot in user.lots) {
            var firstImage = await db.lot_images
                .Where(i => i.lot_id == lot.id)
                .OrderBy(i => i.id)
                .FirstOrDefaultAsync();

            cards.Add(new UserLotCard {
                id = lot.id,
                title = lot.title,
                current_price = lot.current_price,
                thumbnail_path = firstImage?.image_path,
            });
        }
        return cards;
    }

    public string? ValidateBid(Lot? lot, decimal new_price, string user_login)
    {
        if (lot == null)
            return "Лот не найден.";

        if (lot.end_time <= DateTimeOffset.Now)
            return "Лот закрыт для ставок.";

        if (lot.user_login == user_login)

```

```

        return "Вы не можете делать ставку на свой лот.";

    if (new_price <= lot.current_price)
        return "Ставка должна быть больше текущей цены.";

    return null;
}

public async Task<(Lot? lot, string? err)> MakeBid(uint id, decimal
new_price, string user_login)
{
    var lot = await db.lots
        .Include(l => l.user)
        .Include(l => l.bids)
        .FirstOrDefaultAsync(l => l.id == id);

    var error = ValidateBid(lot, new_price, user_login);

    if (lot == null || error != null) {
        return (lot, error);
    }

    var bet = new Bid {
        user_login = user_login,
        lot_id = id,
        price = new_price
    };

    lot.bids.Add(bet);
    lot.current_price = bet.price;
    lot.leader_login = user_login;

    if (!await db.TrySaveChangesAsync())
        return (lot, "Ошибка сохранения ставки.");

    return (await db.lots.Include(l => l.user).Include(l =>
l.bids.OrderByDescending(b => b.id)).FirstOrDefaultAsync(l => l.id == id),
null);
}

public async Task<(Lot? lot, string? err)> CancelLastBid(uint lot_id, string
user_login)
{
    var lot = await db.lots
        .Include(l => l.user)
        .Include(l => l.bids.OrderByDescending(b => b.id))
        .FirstOrDefaultAsync(l => l.id == lot_id);

    if (lot == null) {
        return (lot, "Лот не найден.");
    }

    // TODO: this is not good. change
    var error = ValidateBid(lot, lot.current_price+1, user_login);

    if (error != null) {
        return (lot, error);
    }
}

```



```

var last_2_bid = lot.bids.Take(2).ToList();
if (last_2_bid.Count == 0) {
    return (lot, "Ставок еще не сделано.");
}

var last_bid = last_2_bid.FirstOrDefault();

if (last_bid?.user_login != user_login) {
    return (lot, "Нелегальная операция.");
}

lot.bids.Remove(last_bid);
lot.RecalculatePrice();

if (last_2_bid.Count > 1) {
    lot.leader_login = last_2_bid.Last().user_login;
} else {
    lot.leader_login = null;
}
if (!await db.TrySaveChangesAsync())
    return (null, "Ошибка сохранения ставки.");

return (await db.lots.Include(l => l.user).Include(l =>
l.bids.OrderByDescending(b => b.id)).FirstOrDefaultAsync(l => l.id == lot_id),
null);
}
}

```

Листинг 15 – Файл launchSettings.json

```

{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:8080",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:8081;http://localhost:8080",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}

```

Листинг 16 – Файл password_hasher.cs

```

using System.Security.Cryptography;
namespace App.Services;

public class PasswordHasher
{
    private const int SALT_SIZE = 16;
    private const int HASH_SIZE = 32;
    private const int ITERATIONS = 100000;

    private readonly HashAlgorithmName algorithm = HashAlgorithmName.SHA256;

    public string Hash(string password)
    {
        byte[] salt = RandomNumberGenerator.GetBytes(SALT_SIZE);
        byte[] hash = Rfc2898DeriveBytes.Pbkdf2(password, salt, ITERATIONS,
algorithm, HASH_SIZE);
        return $"{Convert.ToBase64String(hash)}-{Convert.ToBase64String(salt)}";
    }

    public bool Varify(string password, string password_hash)
    {
        string[] pair = password_hash.Split('-');
        byte[] hash = Convert.FromBase64String(pair[0]);
        byte[] salt = Convert.FromBase64String(pair[1]);
        byte[] input_hash = Rfc2898DeriveBytes.Pbkdf2(password, salt,
ITERATIONS, algorithm, HASH_SIZE);
        return CryptographicOperations.FixedTimeEquals(hash, input_hash);
    }
}

```

Листинг 17 – Файл jwt_token_provider.cs

```

using System.Security.Claims;
using System.Text;
using App.Models;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;

namespace App.Services;

public class JwtTokenService(IConfiguration configuration) : BackgroundService
{
    private JsonWebTokenHandler handler = new JsonWebTokenHandler();
    private static List<string> refresh_tokens = new();

    protected override async Task ExecuteAsync(CancellationToken stopping_token)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromHours(1));
        while (!stopping_token.IsCancellationRequested) {
            if (await timer.WaitForNextTickAsync(stopping_token)) {
                await UpdateRefreshTokens();
            }
        }
    }

    public async Task UpdateRefreshTokens()
    {
        var indecies_to_remove = new List<int>();
    }
}

```

```

        foreach (var (i, t) in refresh_tokens.Index()) {
            var res = await handler.ValidateTokenAsync(t,
MakeTokenValidationParameters(configuration));
            if (!res.IsValid) {
                indecies_to_remove.Add(i);
            }
        }
        for (var i = indecies_to_remove.Count-1; i >= 0; i -= 1) {
            refresh_tokens.RemoveAt(i);
        }
        G.Log(LogLevel.Info, "Refresh tokens updated");
    }

    public static SecurityTokenDescriptor MakeTokenDescriptor(IConfiguration
configuration, User user, DateTime expires)
    {
        string secret_key = configuration["Jwt:Secret"]!;
        var security_key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret_key));

        var credentials = new SigningCredentials(security_key,
SecurityAlgorithms.HmacSha256);

        var token_descriptor = new SecurityTokenDescriptor {
            Subject = new ClaimsIdentity([
                new Claim(JwtRegisteredClaimNames.Sub, user.login),
                new Claim(JwtRegisteredClaimNames.Email, user.email ?? ""),
                new Claim("user_login", user.login),
            ]),
            Expires = expires,
            SigningCredentials = credentials,
            Issuer = configuration["Jwt:Issuer"],
        };

        return token_descriptor;
    }

    public static TokenValidationParameters
MakeTokenValidationParameters(IConfiguration configuration)
    {
        return new TokenValidationParameters {
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(configuration["Jwt:Secret"]!)),
            ValidIssuer = configuration["Jwt:Issuer"],
            // ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidateAudience = false,
            ValidateIssuer = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ClockSkew = TimeSpan.Zero
        };
    }

    public (string token, string refresh) GenerateJwtToken(User user)
    {
        var token_descriptor = MakeTokenDescriptor(configuration, user,
DateTime.UtcNow.AddHours(2));
        var token = handler.CreateToken(token_descriptor);
    }

```

```

        token_descriptor.Expires = DateTime.UtcNow.AddHours(2);
        var refresh = GenerateRefreshToken(token_descriptor);

        return (token, refresh);
    }

    public string GenerateRefreshToken(SecurityTokenDescriptor token_descriptor)
    {
        var refresh = handler.CreateToken(token_descriptor);
        refresh_tokens.Add(refresh);
        G.Log(LogLevel.Info, $"Refresh token added: count = {refresh_tokens.Count}");
        return refresh;
    }

    public void AppendTokenCookie(IResponseCookies cookies, User user)
    {
        (string token, string refresh) = GenerateJwtToken(user);

        var options = new CookieOptions{
            HttpOnly = true,
            SameSite = SameSiteMode.Strict,
            Expires = DateTimeOffset.UtcNow.AddMinutes(60),
        };
        cookies.Append("jwt_token", token, options);

        options.Expires = DateTimeOffset.UtcNow.AddHours(2);
        cookies.Append("jwt_refresh_token", refresh, options);
    }

    public async Task<string?> ValidateRefreshToken(string token, string refresh) {
        var validate_params = MakeTokenValidationParameters(configuration);

        var stored_refresh = refresh_tokens.Find(r => r == refresh);
        if (stored_refresh == null) {
            G.Log(LogLevel.Warning, "Refresh token not found in store");
            foreach (var t in refresh_tokens) {
                Console.WriteLine(t);
            }
            return null;
        }

        var refresh_validated = await handler.ValidateTokenAsync(stored_refresh, validate_params);
        if (!refresh_validated.IsValid) {
            G.Log(LogLevel.Warning, "Refresh token is invalid");
            return null;
        }

        var refresh_login = (string?)refresh_validated.Claims["user_login"];
        if (refresh_login == null) {
            G.Log(LogLevel.Warning, "Not user_login in refresh token");
            return null;
        }

        var token_login =

```

```

handler.ReadJsonWebToken(token).Claims.FirstOrDefault(c => c.Type ==
"user_login");
    if (token_login == null) {
        G.Log(LogLevel.Warning, "No user_login in jwt token");
        return null;
    }

    if (token_login.Value != refresh_login) {
        G.Log(LogLevel.Warning, $"User logins not match: from jwt_token =
{token_login.Value}, from refresh token = {refresh_login}");
        return null;
    }

    return token_login.Value;
}

public void RemoveRefreshToken(string refresh)
{
    refresh_tokens.Remove(refresh);
    G.Log(LogLevel.Info, $"Refresh token removed: count =
{refresh_tokens.Count}");
}
}

```

Листинг 18 – Файл lot_updater.cs

```

using Microsoft.EntityFrameworkCore;
using App.Models;
namespace App.Services;

public class AuctionClosingService(IServiceScopeFactory scope_factory) :
BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stopping_token)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromSeconds(10));
        while (await timer.WaitForNextTickAsync(stopping_token))
        {
            await CheckAndCloseAuctions(stopping_token);
        }
    }

    public async Task CheckAndCloseAuctions(CancellationToken stopping_token)
    {
        using var scope = scope_factory.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<AuctionDbContext>();

        try {
            var now = DateTime.UtcNow;
            var expired = await db.lots
                .Where(l => !l.closed)
                .ToListAsync();

            foreach (var l in expired) {
                l.UpdateClosed();
            }

            await db.SaveChangesAsync();
        }
    }
}

```

```

        } catch (Exception e) {
            G.Log(LogLevel.Error, e.Message);
        }
    }
}

```

Листинг 19 – Файл orphan_killer.cs

```

using Microsoft.EntityFrameworkCore;
using App.Models;
namespace App.Services;

public class OrphanImagesKiller : BackgroundService
{
    private string uploads_path = null!;
    private IServiceScopeFactory scope_factory;
    private IWebHostEnvironment env;

    public OrphanImagesKiller(IServiceScopeFactory scope_factory,
    IWebHostEnvironment env) {
        this.scope_factory = scope_factory;
        this.env = env;
        uploads_path = Path.Combine(env.WebRootPath, "uploads", "lot_images");
        Directory.CreateDirectory(uploads_path);
    }

    protected override async Task ExecuteAsync(CancellationToken stopping_token)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(60));
        while (await timer.WaitForNextTickAsync(stopping_token))
        {
            await FindAndKill(stopping_token);
        }
    }

    public async Task FindAndKill(CancellationToken stopping_token)
    {
        try {
            using var scope = scope_factory.CreateScope();
            var db =
scope.ServiceProvider.GetRequiredService<AuctionDbContext>();

            var files = System.IO.Directory.GetFiles(uploads_path).Select(f =>
Path.GetFileName(f));

            var images_to_delete = new List<string>();
            var images = await db.lot_images.Select(i =>
Path.GetFileName(i.image_path)).ToListAsync();

            foreach (var file in files) {
                if (!images.Contains(file)) {
                    images_to_delete.Add(file);
                    G.Log(LogLevel.Info, $"Deleting orphan image: {file}");
                }
            }

            foreach (var file_to_delete in images_to_delete) {
                var file_path = Path.Combine(uploads_path, file_to_delete);

```

```

        if (System.IO.File.Exists(file_path)) {
            System.IO.File.Delete(file_path);
        }
    }
} catch (Exception e) {
    G.Log(LogLevel.Error, e.Message);
}
}
}

```

Листинг 20 – Файл view_path.cs

```

namespace App.Helpers;

public class ViewPath {
    public string path_prefix = "";
    public string file_extention = ".cshtml";

    public string GetPath(string name) {
        return Path.GetFullPath($"{path_prefix}/{name}{file_extention}");
    }
}

```

Листинг 21 – Файл global.cs

```

using System.Diagnostics;
using System.Diagnostics.CodeAnalysis;
namespace App;

public enum LogLevel {
    Debug,
    Info,
    Warning,
    Error,
    Fatal,
}

public static class G
{
    public static IEnumerable<T> EnumIterate<T>()
    {
        return Enum.GetValues(typeof(T)).Cast<T>();
    }

    [DoesNotReturn]
    public static void Todo(string? message = null)
    {
        throw new NotImplementedException($"TODO: {message ?? "no message"}");
    }

    [DoesNotReturn]
    public static T Todo<T>(string? message = null)
    {
        throw new NotImplementedException($"TODO: {message ?? "no message"}");
    }

    [DoesNotReturn]
    public static void Unreachable(string? message = null)
    {
    }
}

```

```

    {
        throw new UnreachableException($"UNREACHABLE: {message ?? "no
message"}");
    }

    [DoesNotReturn]
    public static T Unreachable<T>(string? message = null)
    {
        throw new UnreachableException($"UNREACHABLE: {message ?? "no
message"}");
    }

    public static void Log(LogLevel level, string message)
    {
        var writer = level switch {
            LogLevel.Debug or LogLevel.Info or LogLevel.Warning => Console.Out,
            LogLevel.Error or LogLevel.Fatal => Console.Error,
            _ => G.Unreachable<TextWriter>(nameof(LogLevel)),
        };
        var level_str = level switch {
            LogLevel.Debug => "DEBUG",
            LogLevel.Info => "INFO",
            LogLevel.Warning => "WARNING",
            LogLevel.Error => "ERROR",
            LogLevel.Fatal => "FATAL",
            _ => G.Unreachable<string>("LogLevel"),
        };
        writer.WriteLine($"{level_str}: {message}");
    }
}

```

Листинг 22 – Файл extentions.cs

```

using App.Models;
using App.Helpers;
using Microsoft.AspNetCore.Mvc.ViewFeatures;
namespace App.Extentions;

public static class MyExtentions
{
    extension(HttpContext c) {
        public User? GetUser() => c.Items["user"] as User;
    }

    public const string HX_REQUEST_HEADER = "HX-Request";
    extension(HttpRequest req) {
        public bool IsHtmx() => req.Headers[HX_REQUEST_HEADER] == "true";
    }

    public const string MAIN_LAYOUT_NAME = "main";
    public static readonly ViewPath layout_path = new ViewPath{ path_prefix = "/
Views/Layouts/" };

    extension(ViewDataDictionary view_data) {
        public void SetLayout(string layout_name = MAIN_LAYOUT_NAME) =>
view_data["layout"] = layout_path.GetPath(layout_name);

        public string GetCurrentPath() => (string?)view_data["current_path"] ??

```



```

"";
    public void SetCurrentPath(string path) => view_data["current_path"] =
path;

    public string GetSavedLocation() =>
Uri.UnescapeDataString((string?)view_data["saved_location"] ?? "");
    public void SaveLocation(string url) => view_data["saved_location"] =
Uri.EscapeDataString(url);

    public void SetUser(User user) => view_data["user"] = user;
    public User? GetUser() => view_data["user"] as User;
}
}

public static class EnumExt
{
    extension (PaymentMethod p) {
        public string GetDescription()
        {
            return p switch {
                PaymentMethod.None => "Не указано",
                PaymentMethod.OnMeeting => "При встрече",
                PaymentMethod.ViaBank => "На карту",
                _ => G.Unreachable<string>(nameof(PaymentMethod)),
            };
        }
    }

    extension (DeliveryPayment d) {
        public string GetDescription()
        {
            return d switch {
                DeliveryPayment.PaidBySeller => "Продавец",
                DeliveryPayment.PaidByCustomer => "Покупатель",
                _ => G.Unreachable<string>(nameof(DeliveryPayment)),
            };
        }
    }

    extension (LotStatus s) {
        public string GetDescription()
        {
            return s switch {
                LotStatus.Playing => "Разыгрывается",
                LotStatus.WaitingPurchaseConfirm => "Ожидание подтверждения",
                LotStatus.Purchased => "Выкуплено",
                LotStatus.Denied => "Отказано",
                _ => G.Unreachable<string>(nameof(LotStatus)),
            };
        }
    }
}
}

```

Листинг 23 – Файл attributes.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

```

```

using Microsoft.AspNetCore.Mvc.ActionConstraints;
using Microsoft.AspNetCore.Mvc.Abstractions;

using App.Models;
using App.Extentions;

namespace App.Attributes;

public class GetUserOpt(GetUserOpt.Opt opt) : ActionFilterAttribute
{
    public enum Opt {
        None,
        Strict,
        Admin,
    }

    public override async Task OnActionExecutionAsync(ActionExecutingContext
context, ActionExecutionDelegate next)
    {
        var db =
context.HttpContext.RequestServices.GetRequiredService<AuctionDbContext>();
        var user = await db.GetUserByClaims(context.HttpContext.User);

        if (opt == Opt.Strict && user == null) {
            context.Result = new UnauthorizedResult();
            return;
        }

        if (opt == Opt.Admin && (user == null || !user.is_admin)) {
            context.Result = new UnauthorizedResult();
            return;
        }

        context.HttpContext.Items["user"] = user;
        context.ActionArguments["user"] = user;
        if (context.Controller is Controller c) {
            c.ViewData["user"] = user;
        }

        await next();
    }
}

public class GetUser : GetUserOpt
{
    public GetUser() : base(GetUserOpt.Opt.None) {}
}

public class GetUserStrict : GetUserOpt
{
    public GetUserStrict() : base(GetUserOpt.Opt.Strict) {}
}

public class GetUserAdmin : GetUserOpt
{
    public GetUserAdmin() : base(GetUserOpt.Opt.Admin) {}
}

```

```

// Supporting only russian for now
public enum Language {
    Ru,
}

// TODO: Introduce language changing
public class Title(string title, Language language = Language.Ru) :
    ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        _ = language;
        if (context.Controller is Controller c) {
            c.ViewData["title"] = $"{Config.APP_NAME} - {title}";
        }
    }
}

public class Htmx(bool required = true) : ActionMethodSelectorAttribute
{
    public override bool IsValidForRequest(RouteContext route_context,
        ActionDescriptor action)
    {
        return required == route_context.HttpContext.Request.IsHtmx();
    }
}

public class HtmxServe : ActionFilterAttribute
{
    public const string HX_REDIRECT_HEADER = "HX-Redirect";

    public override void OnActionExecuted(ActionExecutedContext context)
    {
        if (context.Controller is Controller c) {
            if (c.Request.IsHtmx()) {
                if (context.Result is RedirectResult r) {
                    c.Response.Headers.Append(HX_REDIRECT_HEADER, r.Url);
                    context.Result = new OkResult();
                }
            }
        }
    }

    public override void OnActionExecuting(ActionExecutingContext context)
    {
        if (context.Controller is Controller c) {
            if (!c.Request.IsHtmx()) c.ViewData.SetLayout();
            c.ViewBag.htmx = c.Request.IsHtmx();
        }
    }
}

public class SaveLocation : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        if (context.Controller is Controller c) {
            var query_loc = c.Request.Query["saved_location"].FirstOrDefault();

```

```

        if (query_loc != null) {
            c.ViewData.SaveLocation(query_loc);
        }
    }
}

public class AddViewData : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        if (context.Controller is Controller c) {
            c.ViewData.SetCurrentPath(c.Request.Path.ToString());
            c.ViewData["page"] = c.Request.Query["page"].FirstOrDefault();
        }
    }
}

```

Листинг 24 – Файл login.cshtml

```

@using App.Controllers;
@using App.Extentions;
@{
    const string action = "/auth/login";
    const string register = "/auth/register";
    var last_saved_location = ViewData.GetSavedLocation();
}

<form id="login-form" class="form form-boxed shadow" method="post"
action="@action"
    hx-post="@action"
    hx-target="#error"
>
    <input name="last_saved_location" type="hidden"
value="@last_saved_location">

    <label for="login_or_email">Логин:</label>
    <div class="form-field">
        <input id="login_or_email" name="login_or_email" type="text"
placeholder="aboba69" required
            hx-post="/auth/login/user_exists"
            hx-trigger="input changed delay:200ms"
            hx-target="#error"
        />
    </div>

    <label for="password">Пароль:</label>
    <div class="form-field">
        <input id="password" name="password" type="password" placeholder="123"
required />
    </div>

    <div id="error">
        @await Html.PartialAsync("errors")
    </div>

    <button class="form-submit" type="submit">Войти</button>
    <button class="form-reset" type="reset">Сбросить</button>

```

```

        <p class="text-center"><a class="link-decor" href="@register)?
saved_location=@(last_saved_location)"
        hx-get="@register)?saved_location=@(last_saved_location)"
        hx-target="#login-form"
        hx-swap="outerHTML"
        hx-push-url="true"
        >Регистрация</a></p>
</form>

```

Листинг 25 – Файл main.cshtml

```

<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@(ViewData["title"] ?? "No name")</title>
    <link href="/css/style.css" rel="stylesheet">
    <link id="theme-css" href="/css/theme_dark.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.8/dist/htmx.min.js"
integrity="sha384-/
TgkGk7p307TH7EXJDULgG3Ce1UVolA0FopFekQkkXihi5u/60CvVKyzlW+idaz"
crossorigin="anonymous"></script>
    <script src="/js/main.js"></script>
    <script src="/js/ultra.js"></script>
</head>
<body>
    <header>
        @await Html.PartialAsync((string?)ViewData["header_nav"] ?? "/Views/
Navigations/main.cshtml")
    </header>
    <main id="main-content">
        @RenderBody()
    </main>
    <footer>
    </footer>
</body>
</html>

```

Листинг 26 – Файл user_info.cshtml

```

@using App.Extentions;
@model App.Models.UserInfoData;
@{
    var user = ViewData.GetUser()!;
    var this_user = user != null && user.login == Model.user_login;
}

<div class="content-container">
    <div class="user-info">
        <p>Логин: @Model.user_login</p>
        @if (this_user) {
            <p>Почта: @user.email</p>
        }
    </div>

    @if (this_user) {
        <div><a href="/lots/my/create" class="button">Создать лот</a></div>
    }

```

```

}

<div class="user-tabs">
  <div class="tabs-header">
    <button class="tab-btn active" data-tab="lots"
      hx-get="/user/@(Model.user_login)/lots-list"
      hx-target="#tab-content"
      hx-on:click="activate_tab(this)"
    >
      @(this_user ? "Мои лоты" : "Лоты")
    </button>
    @if (this_user) {
      <button class="tab-btn" data-tab="bets"
        hx-get="/user/@(Model.user_login)/bets-list"
        hx-target="#tab-content"
        hx-on:click="activate_tab(this)"
      >
        Мои ставки
      </button>
      <button class="tab-btn" data-tab="bets"
        hx-get="/user/@(Model.user_login)/purchases-list"
        hx-target="#tab-content"
        hx-on:click="activate_tab(this)"
      >
        Мои покупки
      </button>
    }
  </div>
  <div id="tab-content">
    @await Html.PartialAsync("User/user_lots", Model)
  </div>
</div>
<script>
  function activate_tab(el) {
    ultra.by_selector_all('.tab-btn.active').forEach(el => el.
$class_remove('active'));
    ultra.from_dom(el).$class('active');
  }
</script>
</div>

```

Листинг 27 – Файл user_lots.cshtml

```

@using App.Extentions;
@model App.Models.UserInfoData;
@{
  var user = ViewData.GetUser();
  var this_user = user != null && user.login == Model.user_login;
}

@if (Model.lots.Count == 0) {
  <p>У вас пока нет лотов.</p>
} else {
  <div class="user-list">
    @foreach (var lot in Model.lots) {
      <div class="user-list-item" id="lot-@lot.id">
        <div class="user-item-clickable" onclick="ultra.from_dom(this).
$by_selector('.user-item-info a').click()">

```

```

        <div class="user-item-thumb">
            
        </div>
        <div class="user-item-info">
            <a href="/lots/@lot.id">@lot.title</a>
            <span class="lot-price">@lot.current_price</span>
        </div>
    </div>
    @if (this_user) {
        <div class="user-item-actions">
            <button class="delete-btn"
                hx-delete="/lots/my/@lot.id"
                hx-target="#lot-@lot.id"
                hx-swap="outerHTML"
                hx-confirm="Удалить лот?">
                Удалить
            </button>
            <a href="/lots/my/(@lot.id)/edit" class="edit-btn">
                Редактировать
            </a>
        </div>
    }
</div>
}
</div>
}
}

```

Листинг 28 – Файл images_gallery.cshtml

```

@model List<App.Models.LotImage>;
@{
    var first_image = Model.FirstOrDefault();
}

<div class="gallery">
    <div class="main-image-container">
        
        </div>
        <div class="thumbnails-container">
            @if (Model.Count > 1) {
                foreach (var img in Model) {
                    <div class="thumbnail-item @(img == first_image ? "active" :
""))"
                        data-image="@img.image_path">
                            
                        </div>
                }
            }
        </div>
    </div>
    <script>
        (function () {
            const images = ultra.by_selector_all('.thumbnail-item');
            for (const i of images) {
                i.$on_click(function () {
                    const main_img = ultra.by_id('mainImage');

```

```

        const src = this.dataset.image;
        if (src) {
            main_img.src = src;
            for (const i of images) {
                i.$class_remove('active');
            }
            ultra.from_dom(this).$class('active');
        }
    });
}
})();
</script>
</div>

```

Листинг 29 – Файл errors.cshtml

```

@{
    IEnumerable<string>? errors = null;
    if (ViewData["errors"] is IEnumerable<string> e) {
        errors = e;
    } else if (ViewData["errors"] is string s) {
        errors = new List<string>([s]);
    }
}

@if (errors != null) {
    @foreach (var msg in errors) {
        <span class="form-error">@msg</span>
    }
}

```

Листинг 30 – Файл good_results.cshtml

```

@{
    IEnumerable<string>? results = null;
    if (ViewData["result_messages"] is IEnumerable<string> e) {
        results = e;
    } else if (ViewData["result_messages"] is string s) {
        results = new List<string>([s]);
    }
}

@if (results != null) {
    @foreach (var msg in results) {
        <span class="form-good">@msg</span>
    }
}

```

Листинг 31 – Файл lot_meta.cshtml

```

@using App.Extentions;
@using App.Models;
@model App.Models.Lot;
@{
    var payment_ok = Enum.TryParse(Model.payment_method, out PaymentMethod payment_method);
    var delivery_ok = Enum.TryParse(Model.delivery_payment, out DeliveryPayment delivery_payment);
}

```



```

<div class="lot-meta">
    <p>
        @if (Model.closed) {
            <span class="label">Победитель:</span>
        } else {
            <span class="label">Лидер:</span>
        }
        <span id="current-leader" class="value">
            @if (Model.leader_login != null) {
                <a href="/user/
@Model.leader_login">@Model.leader_login@Html.Raw(".")</a>
            } else {
                @Html.Raw("Нет.")
            }
        </span>
    </p>
    <p>
        @if (Model.closed) {
            <span class="label">Лот закрылся:</span>
        } else {
            <span class="label">Лот закроется:</span>
        }
        <span class="value">@Model.end_time.ToString("g")</span>
    </p>
    <p><span class="label">Местоположения:</span> <span
class="value">@Model.city</span></p>
    <p><span class="label">Начальная цена:</span> <span
class="value">@Model.initial_price ₺</span></p>
    <p><span class="label">Количество:</span> <span class="value">@Model.count</
span></p>
    <br>
    <p><span class="label">Метод оплаты:</span> <span
class="value">@(payment_ok ? payment_method.GetDescription() : "Неизвестно")</
span></p>
    <p><span class="label">Доставка оплачивает:</span> <span
class="value">@(delivery_ok ? delivery_payment.GetDescription() :
"Неизвестно")</span></p>
</div>

```

Листинг 32 – Файл _ViewStart.cshtml

```

@{
    Layout = (string?)ViewData["layout"] ?? "";
}

```

Листинг 33 – Файл lot_card.cshtml

```

@model App.Models.HomePageModel.LotCard;
@{
    var current_page = (string?)ViewData["page"] ?? "0";
}

<div class="lot-card hover-shadow"
    hx-get="/lots/@Model.id?page=@current_page"
    hx-target="main"
    hx-push-url="true"
>
    

```

```

<p>@Model.title</p>
<span class="lot-price">@Model.price</span>
<p class="lot-card-end-time">
  @if (@Model.closed) {
    <span class="lot-card-closed">Закрыт</span>
  } else {
    <span>До</span>
  }
  @Model.end_time.ToString("dd MMM yyyy, hh:mm")
</p>
</div>

```

Листинг 34 – Файл pagination_links.cshtml

```

@model App.Models.HomePageModel;

@if (Model.pages_count > 1) {
  <div class="pagination">
    <button class="page-btn" @(Model.query.page == 0 ? "disabled" : "")
      hx-get="/lots"
      hx-include="#filters-form input"
      hx-target="#lot-cards-container"
      hx-push-url="true"
      hx-vals="js:{...get_query_with('page', @(Model.query.page -
1))}">
      ← Предыдущая
    </button>
    <span class="page-info">Страница @(Model.query.page + 1) из
@Model.pages_count</span>
    <button class="page-btn" @(Model.query.page >= Model.pages_count - 1 ?
"disabled" : "")
      hx-get="/lots"
      hx-include="#filters-form input"
      hx-target="#lot-cards-container"
      hx-push-url="true"
      hx-vals="js:{...get_query_with('page', @(Model.query.page +
1))}">
      Следующая →
    </button>
  </div>
}

```

Листинг 35 – Файл lot_cards_grid.cshtml

```

@model App.Models.HomePageModel;
@{
  var need_pagination = (bool?)ViewData["pagination"] ?? false;
}

<div class="lot-cards-grid">
  @foreach (var card in Model.cards) {
    @await Html.PartialAsync("lot_card", card)
  }
</div>
@if (need_pagination) {
  <div hx-swap-oob="true" id="pagination-links">
    @await Html.PartialAsync("pagination_links", Model)
  </div>
}

```

Листинг 36 – Файл index.cshtml

```
@model App.Models.HomePageModel;
@using App.Models;
@{
    var user = ViewData["user"] as User;
}

<form id="filters-form" class="filters-form">
    <input type="hidden" name="page" value="@Model.query.page" />
    <input type="hidden" name="page_size" value="@Model.query.page_size" />
    <input type="hidden" name="tag_id" value="@Model.query.tag_id" />
    <input type="hidden" name="search" value="@Model.query.search ?? """ />
    <input type="hidden" name="sort_by" value="@Model.query.sort_by" />
    <input type="hidden" name="sort_dir" value="@Model.query.sort_dir" />
    <input type="hidden" name="show_closed" value="@Model.query.show_closed ?
"true" : "false")" />
</form>

<div class="main-layout">
    <aside class="lot-filters">
        <div class="filter-section">
            <p>Категории</p>
            <div class="lot-tags">
                <a href="#" class="tag-link @Model.query.tag_id == null ?
"active" : "")"
                    hx-get="/lots"
                    hx-target="#lot-cards-container"
                    hx-push-url="true"
                    hx-on:click="activate_tag_links(this)"
                    hx-vals="js:{...get_query_with('tag_id', '')}"
                >
                    Bce
                </a>
                @foreach (var tag in Model.tags) {
                    <a href="#" class="tag-link @Model.query.tag_id == tag.id ?
"active" : "")"
                        hx-get="/lots"
                        hx-target="#lot-cards-container"
                        hx-push-url="true"
                        hx-on:click="activate_tag_links(this)"
                        hx-vals="js:{...get_query_with('tag_id', '@tag.id')}"
                    >
                        @tag.name
                    </a>
                }
            </div>
        </div>
    </div>

    <div class="filter-section">
        <p>Фильтры</p>
        <label>
            <input type="checkbox" @(Model.query.show_closed ? "checked" :
"" )
                hx-trigger="change"
                hx-get="/lots"
                hx-target="#lot-cards-container"
                hx-push-url="true"
            >
        </label>
    </div>
</div>
```

```

        hx-vals="js:{...get_query_with('show_closed',
this.checked ? 'true' : 'false'))}" />
        Показывать закрытые
    </label>

    <hr>

    <div class="sort-section">
        <p>Сортировка</p>
        <select class="form-like"
            hx-trigger="change"
            hx-get="/lots"
            hx-target="#lot-cards-container"
            hx-push-url="true"
            hx-vals="js:{...get_query_with('sort_by', this.value)}">
            <option value="end_time" selected="@ (Model.query.sort_by ==
"end_time")">По времени</option>
            <option value="price" selected="@ (Model.query.sort_by ==
"price")">По цене</option>
            <option value="title" selected="@ (Model.query.sort_by ==
"title")">По названию</option>
        </select>
        <select class="form-like"
            hx-trigger="change"
            hx-get="/lots"
            hx-target="#lot-cards-container"
            hx-push-url="true"
            hx-vals="js:{...get_query_with('sort_dir',
this.value)}">
            <option value="asc" selected="@ (Model.query.sort_dir ==
"asc")">По возрастанию</option>
            <option value="desc" selected="@ (Model.query.sort_dir ==
"desc")">По убыванию</option>
        </select>
    </div>

    <hr>

    <label>
        <p>Элементов на странице</p>
        <select class="form-like"
            hx-trigger="change"
            hx-get="/lots"
            hx-target="#lot-cards-container"
            hx-push-url="true"
            hx-vals="js:{...get_query_with('page_size',
this.value)}">
            <option value="15" selected="@ (Model.query.page_size ==
15)">15</option>
            <option value="30" selected="@ (Model.query.page_size ==
30)">30</option>
        </select>
    </label>
</div>
</aside>
<div class="main-content">
    <div class="search-bar">

```

```

        <input class="form-like" type="text" name="search_input"
placeholder="Поиск..." value="@ (Model.query.search ?? "")"
        hx-trigger="input changed delay:500ms"
        hx-get="/lots"
        hx-target="#lot-cards-container"
        hx-push-url="true"
        hx-vals="js:{...get_query_with('search', this.value)}" />
    </div>

    <div id="pagination-links">
        @await Html.PartialAsync("pagination_links", Model)
    </div>

    <div id="lot-cards-container">
        @await Html.PartialAsync("lot_cards_grid", Model)
    </div>
</div>

<script>
    function get_query_with(name, value) {
        ultra.by_selector(`#filters-form input[name="${name}"]`).value =
value;
        const inputs = ultra.by_selector_all('#filters-form input');
        const values = Array.from(inputs).reduce((acc, i) => {
            acc[i.name] = i.value;
            return acc;
        }, {});
        return values;
    }
    function activate_tag_links(element) {
        ultra.by_selector_all('.tag-link.active').forEach(t => t.
$class_toggle('active'));
        element.classList.toggle('active')
    }
</script>
</div>

```

Листинг 37 – Файл tags.cshtml

```

@model IEnumerable<App.Models.TagNavigation>;

<nav class="tags-nav">
    <ul>
        @foreach (var it in @Model) {
            <li>
                <a href="/?tag_id=@(it.id)">@(it.name)</a>
            </li>
        }
    </ul>
</nav>

```

Листинг 38 – Файл main.cshtml

```

@using App.Models;
@using App.Extentions;
@{
    var current_path = ViewData.GetCurrentPath();
    var user = ViewData.GetUser();
}

```

```

<nav class="header-nav">
  <div id="logo">
    <a href="/">АУК</a>
  </div>
  <ul class="header-nav-pages">
    <li class="header-link hover-shadow">
      <a href="/" class="@ (current_path == "/" ? "link-decor" : null)">
        Главная
      </a>
    </li>
    @if (user != null) {
      @if (user.is_admin) {
        <li class="header-link hover-shadow">
          <a href="/admin" class="@ (current_path == "/admin" ? "link-
decor" : null)">
            Админ
          </a>
        </li>
      }
      <li class="header-link hover-shadow">
        <a href="/user" class="@ (current_path == "/user/" + user?.login ?
"link-decor" : null)">
          Аккаунт
        </a>
      </li>
      <li class="header-user-login">
        @user.login
      </li>
      <li class="header-link hover-shadow">
        <a href="/auth/logout?saved_location=@ (current_path)"
          hx-post="/auth/logout?saved_location=@ (current_path)"
        >
          Выход
        </a>
      </li>
    } else {
      <li class="header-link hover-shadow">
        <a href="/auth/login" class="@ (current_path == "/auth/login" ?
"link-decor" : null)">
          Вход
        </a>
      </li>
    }
  </ul>
</nav>

```

Листинг 39 – Файл purchase_form.cshtml

```

@model App.Models.Lot;

<div class="form form-narrow">
  <div class="purchase-info">
    <h1>Покупка лота</h1>
    <h2>@Model.title</h2>
    <p>Конечная цена: <span class="lot-price">@Model.current_price</span></
p>
    <div class="seller-info">

```

```

        <span class="label">Продавец:</span>
        <a href="/user/@Model.user?.login">@(Model.user?.login ??
"Неизвестно")</a>
    </div>
    <hr>
    <h3>Описание</h3>
    @if (@Model.description != null) {
        <p>@Model.description</p>
    } else {
        <p>Описания нет.</p>
    }
    <hr>
    <h2>Фото товара</h2>
    @await Html.PartialAsync("images_gallery", Model.images)

    @await Html.PartialAsync("lot_meta", Model)
</div>

<hr>

<div hx-confirm="Вы уверены?">
    <button type="submit" class="form-submit"
        hx-post="/lots/my/@Model.id/purchase-submit"
    >Согласиться</button>
    <button type="reset" class="form-reset"
        hx-post="/lots/my/@Model.id/purchase-deny"
    >Отклонить</button>
</div>
</div>

```

Листинг 40 – Файл create_form.cshtml

```

@using App;
@using App.Models;
@using App.Extensions;
@model App.Models.CreateFormData;

<form class="form form-narrow" id="lot-create-form"
    action="/lots/my/create" method="post"
    enctype="multipart/form-data"
>
    <label for="name">Название</label>
    <div class="form-field">
        <input id="title" name="title" type="text" required autofocus />
    </div>

    <label for="city">Город</label>
    <div class="form-field">
        <input id="city" name="city" type="text" required />
    </div>

    <label for="price">Цена</label>
    <div class="form-field">
        <input id="price" name="price" type="number" required />
    </div>

    <label for="count">Количество</label>
    <div class="form-field">

```

```

        <input id="count" name="count" type="number" step="1" value="1" min="1"
required/>
    </div>

    <label for="tag">Категория</label>
    <div class="form-field">
        <select name="tag_id" id="tag" required>
            <option value="" selected>Выберите</option>
            @foreach(var tag in @Model.tags) {
                <option value="@tag.id">@tag.name</option>
            }
        </select>
    </div>

    <label for="end_time">Дата окончания</label>
    <div class="form-field">
        <input id="end_time" name="end_time" type="datetime-local" required />
    </div>

    <label for="description">Описание</label>
    <div class="form-field">
        <textarea id="description" name="description" type="text"></textarea>
    </div>

    <label for="thumbnail">Обложка</label>
    <div class="form-field">
        <input id="thumbnail-input" name="thumbnail" type="file" accept="image/
*" />
        <div id="thumbnail-container" class="file-loader"></div>
    </div>

    <div class="file-load-toolbar">
        <label for="images">Фото товара</label>
        <button type="button" class="form-button" id="add-files">Добавить
файлы</button>
        <button type="button" class="form-reset" id="clear-files">Очистить
файлы</button>
        <div class="form-field file-loader">
            <input id="files-input" name="images" type="file" multiple
accept="image/*" hidden />
            <div id="file-container" data-state="files" class="image-cards-
container"></div>
            <script src="/js/files_load_toolbar.js"></script>
        </div>
    </div>

    <label for="payment_method">Метод оплаты</label>
    <div class="form-field">
        <select id="payment_method" name="payment_method" required>
            <option value="" selected>Выберите</option>
            @foreach (var p in G.EnumIterate<PaymentMethod>()) {
                <option value="@p.ToString()">@p.GetDescription()</option>
            }
        </select>
    </div>

    <label for="delivery_payment">Оплата доставки</label>
    <div class="form-field">

```



```

        <select id="delivery_payment" name="delivery_payment" required>
            <option value="" selected>Выберите</option>
            @foreach (var p in G.EnumIterate<DeliveryPayment>()) {
                <option value="@p.ToString()">@p.GetDescription()</option>
            }
        </select>
    </div>

    <div id="error">
        @await Html.PartialAsync("errors")
    </div>

    <hr>

    <button class="form-submit" type="submit">Создать</button>
    <button class="form-reset" type="reset">Очистить</button>

    <script src="/js/lot_form.js"></script>
    <script src="/js/lot_create.js"></script>
</form>

```

Листинг 41 – Файл edit_form.cshtml

```

@using App;
@using App.Models;
@using App.Extentions;

@model App.Models.CreateFormData;

<form class="form form-narrow" id="lot-edit-form"
    enctype="multipart/form-data"

    hx-patch="/lots/my/@(Model.current_lot.id)"
    hx-target="#results"
    hx-swap="innerHTML"
>
    <label for="name">Название</label>
    <div class="form-field">
        <input id="title" name="title" type="text" required autofocus
value="@Model.current_lot.title" />
    </div>

    <label for="city">Город</label>
    <div class="form-field">
        <input id="city" name="city" type="text" required
value="@Model.current_lot.city" />
    </div>

    <label for="count">Количество</label>
    <div class="form-field">
        <input id="count" name="count" type="number" step="1" min="1" required
value="@Model.current_lot.count" />
    </div>

    <label for="tag">Категория</label>
    <div class="form-field">
        <select name="tag_id" id="tag" required>
            @{

```

```

        var selected_tag_id = Model.current_lot.tag_id;
        var select_tag = (uint tag_id) => tag_id == selected_tag_id ?
"selected" : "";
    }
    <option value="">Выберите</option>
    @foreach(var tag in @Model.tags) {
        <option value="@tag.id" @(select_tag(tag.id))>@tag.name</option>
    }
    </select>
</div>

<label for="end_time">Дата окончания</label>
<div class="form-field">
    <input id="end_time" name="end_time" type="datetime-local" required
value="@((Model.current_lot.end_time.ToString("yyyy-MM-ddThh:mm")))" />
</div>

<label for="description">Описание</label>
<div class="form-field">
    <textarea id="description" name="description"
type="text">@Model.current_lot.description</textarea>
</div>

<label for="thumbnail">Обложка</label>
<div class="form-field">
    <input id="thumbnail-input" name="thumbnail" type="file" accept="image/*" />
    <div id="thumbnail-container" class="file-loader"></div>
</div>

<div class="file-load-toolbar">
    <label for="images">Фото товара</label>
    <button type="button" class="form-button" id="add-files">Добавить
файлы</button>
    <button type="button" class="form-reset" id="clear-files">Очистить
файлы</button>
    <div class="form-field file-loader">
        <input id="files-input" name="images" type="file" multiple
accept="image/*" hidden />
        <div id="file-container" data-state="files" class="image-cards-
container"></div>
    </div>
    <script src="/js/files_load_toolbar.js"></script>
    <script src="/js/load_images_for_edit.js"></script>
</div>

<label for="payment_method">Метод оплаты</label>
<div class="form-field">
    <select id="payment_method" name="payment_method" required>
        @{
            var selected_payment = Model.current_lot.payment_method;
            var select_payment = (string pm) => pm == selected_payment ?
"selected" : "";
        }
        <option value="" @(string.IsNullOrEmpty(selected_payment)) ?
"selected" : "">Выберите</option>
        @foreach (var p in G.EnumIterate<PaymentMethod>()) {
            <option value="@p.ToString()"

```

```

@ (select_payment(p.ToString()))>@p.GetDescription()</option>
    }
  </select>
</div>

<label for="delivery_payment">Оплата доставки</label>
<div class="form-field">
  <select id="delivery_payment" name="delivery_payment" required>
    @{
      var selected_delivery = Model.current_lot.delivery_payment;
      var select_delivery = (string dm) => dm == selected_delivery ?
"selected" : "";
    }
    <option value="" @(string.IsNullOrEmpty(selected_delivery) ?
"selected" : "")>Выберите</option>
    @foreach (var p in G.EnumIterate<DeliveryPayment>()) {
      <option value="@p.ToString()"
@ (select_delivery(p.ToString()))>@p.GetDescription()</option>
    }
  </select>
</div>

<div id="results"></div>

<hr>

<button class="form-submit" type="submit" >Сохранить</button>
<button class="form-reset" type="reset">Вернуть</button>

<script src="/js/lot_form.js"></script>
<script src="/js/lot_edit.js"></script>
<script>
  (async function() {
    await load_images(
      @foreach (var image in Model.current_lot.images.Skip(1)) {
        @Html.Raw($"' {image.image_path}',")
      }
    );

    const thumbnail_path =
'@Model.current_lot.images.FirstOrDefault()?.image_path';
    const thumbnail = await files_toolbar.url_to_file(thumbnail_path,
thumbnail_path.split('/').at(-1));
    const dt = new DataTransfer();
    dt.items.add(thumbnail);
    const thumbnail_input = ultra.by_id('thumbnail-input');
    thumbnail_input.files = dt.files;
    thumbnail_input.dispatchEvent(new Event('change'));
  })();
</script>

</form>

```

Листинг 42 – Файл bid_history.cshtml

```

@model ICollection<App.Models.Bid>;

<h3>История ставок</h3>

```

```

@if (Model.Count <= 0) {
    <p>Ставки пока никто не делал.</p>
} else {
    <div class="bid-history-list">
        @foreach (var (index, bid) in Model.OrderByDescending(b =>
b.id).Index()) {
            <div class="bid-history-item">
                <span class="bid-history-item-index @(index == 0 ? "highlight" :
"" )">@(index+1).</span>
                <a class="bid-history-item-user @(index == 0 ? "highlight" :
"" )" href="/user/@(bid.user_login)">@(bid.user_login)</a>
                <span class="lot-price @(index == 0 ? "highlight" :
"" )">@(bid.price)</span>
            </div>
        }
    </div>
}

```

Листинг 43 – Файл bid_maker.cshtml

```

@using App.Extentions;
@model App.Models.LotDetailsViewModel;
@{
    var min_bid_str = (Model.lot.current_price + 0.01M).ToString("0.00");
    var user = ViewData.GetUser();
}

<div id="bid-maker">
    @if (Model.lot.closed) {
        <p class="bid-notice">Лот закрыт.</p>
    } else if (Model.is_owner) {
        <p class="bid-notice">Вы владелец этого лота.</p>
    } else {
        if (user != null) {
            if (@Model.lot.bids.Count > 0 && @Model.lot.leader_login ==
user.login) {
                <p class="bid-notice">Вы уже сделали ставку.</p>
                <p>Вы сможете сделать ее повторно, если кто-то ее перебьет или
отмените ее.</p>
                <button class="button"
                    hx-delete="/lots/@(Model.lot.id)/bid"
                    hx-target="#bid-maker"
                    hx-swap="innerHTML"
                >
                    Отменить ставку
                </button>
            } else {
                <form class="form bid-form" method="post" action="/lots/
@(Model.lot.id)/bid"
                    hx-post="/lots/@(Model.lot.id)/bid"
                    hx-target="#bid-maker"
                    hx-swap="outerHTML"
                >
                    <div class="form-field">
                        <input id="bid-input" type="number" name="new_price"
placeholder="Ваша ставка" step="0.01" min="@ (min_bid_str)"
value="@ (min_bid_str)" required />
                        <span class="lot-price"></span>

```

```

        </div>
        <button class="form-submit" type="submit">Сделать ставку</
button>
        </form>
        <div id="error">
            @await Html.PartialAsync("errors")
        </div>
    }
    } else {
        <p class="bid-notice">
            Войдите, чтобы делать ставки.
            <a href="/auth/login?savе_location=@ViewData.GetCurrentPath()"
class="button">Вход</a>
        </p>
    }
}
</div>
@if (@Model.price_changed) {
    <span hx-swap-oob="true" id="lot-price" class="lot-
price">@Model.lot.current_price</span>
    <div hx-swap-oob="true" id="bid-history" class="bid-history">
        @await Html.PartialAsync("bid_history", @Model.lot.bids)
    </div>
    @if (Model.lot.leader_login != null) {
        <span hx-swap-oob="true" id="current-leader" class="value"><a href="/
user/@(Model.lot.leader_login)">@Model.lot.leader_login</a></span>
    } else {
        <span hx-swap-oob="true" id="current-leader" class="value">Нет</span>
    }
}
}

```

Листинг 44 – Файл lot_details.cshtml

```

@using App.Models;
@using App.Extentions;
@model App.Models.LotDetailsViewModel;
@{
    var saved_page = (string?)ViewData["page"] ?? "0";
    var user = ViewData.GetUser();
}

<div class="content-container lot-detail">
    <a href="/?page=@saved_page" class="back-link"><- Вернуться</a>

    <div class="lot-detail-grid">
        @await Html.PartialAsync("images_gallery", Model.lot.images)

        <div class="info">
            <h1>@Model.lot.title</h1>

            <div class="lot-price-details">
                <span class="lot-price-label">Текущая цена:</span>
                <span id="lot-price" class="lot-
price">@Model.lot.current_price</span>
            </div>

            @await Html.PartialAsync("bid_maker", Model)

```

```

        @if (Model.lot.closed && Model.lot.leader_login != null && user !=
null && user.login == Model.lot.leader_login) {
            @if (Model.lot.status ==
LotStatus.WaitingPurchaseConfirm.ToString()) {
                <div id="purchase-button">
                    <p style="margin-top:0;">Вы победили! Можете оставить
заказ, чтобы забрать содержимое лота.</p>
                    <p style="margin-top:30px;"><a class="button" href="/
lots/my/@(Model.lot.id)/purchase">Оставить заказ</a></p>
                </div>
            } else if (Model.lot.status == LotStatus.Purchased.ToString()) {
                <p style="margin-top:0;">Вы выкупили лот.</p>
            } else if (Model.lot.status == LotStatus.Denied.ToString()) {
                <p style="margin-top:0;">Вы отказались от покупки.</p>
            }
        }

        @await Html.PartialAsync("lot_meta", Model.lot)

        <div class="lot-description">
            <h3>Описание</h3>
            @if (Model.lot.description != null) {
                <p>@Model.lot.description</p>
            } else {
                <p>Нет описания</p>
            }
        </div>

        <div class="seller-info">
            <span class="label">Продавец:</span>
            <a href="/user/@Model.seller?.login">@(Model.seller?.login ??
"Неизвестно")</a>
            @if (user != null && Model.lot.user_login == user.login) {
                <p>Вы продавец.</p>
                <a href="/lots/my/@Model.lot.id/edit" class="edit-
btn">Редактировать</a>
            }
        </div>

    </div>
</div>
<div id="bid-history" class="bid-history">
    @await Html.PartialAsync("bid_history", Model.lot.bids)
</div>
</div>

```

Листинг 45 – Файл user_purchases.cshtml

```

@model List<App.Models.Purchase>;

@if (Model.Count <= 0) {
    <p>Вы ещё не делали покупок.</p>
} else {
    <div class="user-list">
        @foreach (var purchase in Model) {
            <div class="user-list-item" id="lot-@purchase.lot.id">
                <div class="user-item-thumb">
                    
</div>
<div class="user-item-info">
    <a href="/lots/@purchase.lot.id">@purchase.lot.title</a>
    <span class="lot-price">@purchase.lot.current_price</span>
</div>
</div>
}
</div>
}

```

Листинг 46 – Файл user_bets.cshtml

```

@using App.Extentions;
@using App.Models;
@model List<App.Models.Bid>
@{
    var rendered_lots = new List<uint>();
    var user = ViewData.GetUser()!;
}

@if (Model.Count == 0) {
    <p>Вы ещё не делали ставок.</p>
} else {
    <div class="user-list">
        @foreach (var bet in Model) {
            @if (rendered_lots.Contains(bet.lot.id)) {
                continue;
            } else {
                rendered_lots.Add(bet.lot.id);
            }
            <div class="user-list-item" onclick="ultra.from_dom(this).
$by_selector('.user-item-info a').click()">
                <div class="user-item-thumb">
                    
                </div>
                <div class="user-item-info">
                    <a href="/lots/@bet.lot.id">@bet.lot.title</a>
                </div>
                @if (bet.lot.leader_login == user.login) {
                    @if (bet.lot.closed) {
                        @if (bet.lot.status ==
LotStatus.WaitingPurchaseConfirm.ToString()) {
                            <span class="leader-badge">Вы победили</span>
                        } else if (bet.lot.status ==
LotStatus.Purchased.ToString()) {
                            <span class="leader-badge">Выкуплен</span>
                        } else {
                            <span class="outbid-badge">Отказ</span>
                        }
                    } else {
                        <span class="leader-badge">Вы лидер</span>
                    }
                } else {
                    <div class="outdib">
                        <span class="outbid-badge">Вас перебили</span>
                    }
                }
            }
        }
    }

```

```

        <span class="outbid-price">Текущая цена: </span>
        <span class="lot-price">@bet.lot.current_price</span>
    </div>
}
<div class="user-item-price">
    <span class="price-label">Ваша ставка: </span>
    <span class="lot-price">@bet.price</span>
</div>
<div class="user-item-end">
    Окончание: @bet.lot.end_time.ToString("g")
</div>
</div>
}
</div>
}

```

Листинг 47 – Файл tag_item.cshtml

```

@model App.Models.Tag

<div class="user-list-item" id="tag-row-@Model.id">
    <div class="user-item-info" style="flex:1;">
        <span style="flex:1;">@Model.id. @Model.name</span>
    </div>
    <div class="user-item-actions">
        <button class="edit-btn" hx-get="/admin/@Model.id/edit-form"
            hx-target="#tag-row-@Model.id"
            hx-swap="outerHTML">
            Редактировать
        </button>
        <button class="delete-btn" hx-delete="/admin/@Model.id"
            hx-target="#tag-row-@Model.id"
            hx-swap="outerHTML"
            hx-confirm="Удалить категорию?">
            Удалить
        </button>
    </div>
</div>

```

Листинг 48 – Файл tag_list.cshtml

```

@model App.Models.AdminPageModel

@if (Model.Tags.Count == 0) {
    <p>Категории не найдены.</p>
} else {
    <div class="user-list tag-admin-list">
        @foreach (var tag in Model.Tags) {
            @await Html.PartialAsync("tag_item", tag)
        }
    </div>
}

```

Листинг 49 – Файл index.cshtml

```

@using App.Models;
@model App.Models.AdminPageModel;

<div class="content-container">

```



```

<h1>Управление категориями</h1>

<div class="admin-controls">
  <div class="search-bar">
    <input class="form-like" type="text" id="admin-search"
placeholder="Поиск категорий..."
    value="@Model.Search"
    hx-get="/admin"
    hx-trigger="input changed delay:300ms"
    hx-target="#tag-list-container"
    hx-push-url="true"
    hx-vals='js:{search: this.value}' />
    <button class="button" id="show-create-form">Создать категорию</
button>
  </div>
</div>

<div id="tag-form-container" style="display: none;">
  @await Html.PartialAsync("tag_form", Model)
</div>

<div id="tag-list-container">
  @await Html.PartialAsync("tag_list", Model)
</div>
</div>

<script>
  document.getElementById('show-create-form').addEventListener('click',
function() {
    const container = document.getElementById('tag-form-container');
    if (container.style.display === 'none') {
      container.style.display = 'block';
      this.textContent = 'Скрыть форму';
    } else {
      container.style.display = 'none';
      this.textContent = 'Создать категорию';
    }
  });
</script>

```

Листинг 50 – Файл tag_form.cshtml

```

@model App.Models.AdminPageModel

@if (Model.Success) {
  <div id="tag-list-container" hx-swap-oob="true">
    @await Html.PartialAsync("tag_list", Model)
  </div>
}

<form class="form" method="post" action="/admin"
  hx-post="/admin"
  hx-target="this"
  hx-swap="outerHTML"
  hx-on::after-request="this.reset()">
  <div class="form-field">
    <label for="tag-name">Название категории</label>
    <input class="form-like" id="tag-name" name="name" type="text" required/

```

```

>
</div>
<button class="form-submit" type="submit">Создать</button>
<div id="create-errors">
    @if (Model.Success) {
        @await Html.PartialAsync("good_results")
    } else {
        @await Html.PartialAsync("errors")
    }
</div>
</form>

```

Листинг 51 – Файл tag_edit_form.cshtml

```

@model App.Models.TagEditForm

<form class="form" method="post" action="/admin/@Model.TagId"
    hx-patch="/admin/@Model.TagId"
    hx-target="this"
    hx-swap="outerHTML">
    <div class="form-field" style="display:flex; gap:10px; align-items:center;">
        <input class="form-like" name="name" type="text" required
            value="@Model.Name" />
        <button class="form-submit" type="submit">Сохранить</button>
        <button class="form-reset" type="button"
            hx-get="/admin?search=@(ViewBag.Search ?? "")"
            hx-target="#tag-row-@Model.TagId"
            hx-swap="outerHTML">
            Отмена
        </button>
    </div>
    <div id="edit-errors">
        @await Html.PartialAsync("errors")
    </div>
</form>

```